УЧЕБНОЕ ПОСОБИЕ · ПОДГОТОВКА К ТЕСТАМ И ЭКЗАМЕНУ

Информатика: программирование на C Недели 4–7

Все правила, вопросы и задания из лекций, лабораторий и bug hunt-ов курса — с модельными ответами, разбором кода и решениями для каждой недели: типы данных, управляющие конструкции, функции, структуры, указатели, строки и рекурсия.

Курс Computer Science (C Programming) · Fall semester 2026
Ташкентский технологический университет

SAVOLLAR · TESTLAR · KOD · QOIDALAR

ОКТЯБРЬ 2026

Содержание

1. Карта курса, источники и как пользоваться пособием . . . 3

1.1. Недели 4–7 и файлы курса . . . 3

2. Неделя 4 — Типы данных и операторы (Data Types and Operators) . . . 4

2.1. Ключевые положения («все правила» недели 4) . . . 4

2.2. Savollar недели 4 — вопросы лекции и лаборатории с модельными ответами . . . 9

2.3. Testlar недели 4 . . . 11

2.4. Atamalar — глоссарий недели 4 . . . 13

3. Неделя 5 — Управляющие конструкции (Control Structures) . . . 13

3.1. Ключевые положения («все правила» недели 5) . . . 14

3.2. Savollar недели 5 — вопросы лекции и лаборатории с модельными ответами . . . 19

3.3. Testlar недели 5 . . . 21

3.4. Atamalar — глоссарий недели 5 . . . 23

4. Неделя 6 — Функции и структуры (Functions and Structures) . . . 23

4.1. Ключевые положения («все правила» недели 6) . . . 24

4.2. Savollar недели 6 — вопросы лекции и лаборатории с модельными ответами . . . 30

4.3. Testlar недели 6 . . . 32

4.4. Atamalar — глоссарий недели 6 . . . 33

5. Неделя 7 — Указатели, строки и рекурсия (Pointers, Strings, Recursion) . . . 34

5.1. Ключевые положения («все правила» недели 7) . . . 34

5.2. Savollar недели 7 — вопросы лекции и лаборатории с модельными ответами . . . 41

5.3. Testlar недели 7 . . . 43

5.4. Atamalar — глоссарий недели 7 44

6. Bug Hunts — все ошибки и их исправления 45

6.1. Bug Hunt C 45

6.2. Bug Hunt Loops 50

6.3. Bug Hunt Pointers & Strings 55

7. Шпаргалка: ключевые правила и таблицы 59

7.1. Приоритет операторов 59

7.2. Форматы printf/scanf 60

7.3. Ключевые правила C 61

7.4. Десять ответов одной строкой 62

7.5. Что дальше 62

1. Карта курса, источники и как пользоваться пособием

Это учебное пособие собрано по всем файлам курса **Computer Science — программирование на C** (трек **Foundations of Computer Science, FoCS**; осенний семестр 2026, Ташкентский технологический университет). Курс устроен из четырёх слоёв: (1) еженедельные лекции — FoCS/TashTech Lectures 4–7 (6, 13, 20 и 27 октября 2026) — повторение предыдущей темы и новая тема; (2) лабораторные работы по главам 5–8 учебника Ceri, Mandrioli & Sbattella «The Art and Craft of Computing» в C11-адаптации с помощниками ввода CS50 (get_{double}, get_{char}, get_{string}); (3) рабочие листы **bug hunt** — «охота за ошибками» по условным конструкциям и функциям, циклам, указателям и строкам; (4) проектные файлы **кофемашины** (классная модель недели 6: Stock, Recipe, Cup). Пособие охватывает материал недель 4–7: типы данных и операторы, управляющие конструкции, функции и структуры, указатели, строки и рекурсию.

Каждый савол (savollar — вопросы) из лекций и лабораторий, каждый тест (testlar), каждый атама (atamalar — термины) и каждый фрагмент кода из слайдов, лабораторий и bug hunt-ов выписан полностью и снабжён модельным ответом или решением. Все примеры кода взяты из файлов курса дословно; где листинг PDF-извлечения был «побит» пробелами, он восстановлен по смыслу слайда. Оригинальные формулировки преподавателя сохранены: «Pop quiz», «Prediction», «Exit check», «Expected».

Как пользоваться пособием: (1) начните с раздела «Ключевые положения» каждой недели — это «все правила» (qoidalar) недели в сжатом виде с таблицами; (2) прорешайте открытые вопросы (savollar) вслух, закрыв ответы, и сверьтесь с модельными ответами; (3) пройдите все тесты (testlar) — на контрольных используются именно такие формулировки; (4) разберите весь код построчно: на экзамене спрашивают «что напечатает эта программа?»; (5) выучите глоссарий atamalar — термины встречаются в вопросах на определение; (6) перед экзаменом повторите все bug hunt-ы из главы 6 и шпаргалку из главы 7 (приоритет операторов, форматы printf/scanf, десять ответов одной строкой).

1.1. Недели 4–7 и файлы курса

Каждой неделе соответствует набор файлов в папке курса; все они переработаны в этом пособии. Лекции идут по вторникам, лаборатории — по главам учебника (глава 5 — неделя 4, глава 6 — неделя 5, глава 7 — неделя 6, глава 8 — неделя 7).

Неделя	Тема	Лекции/лаборатория	Доп. файлы
4	Типы данных и операторы (Data Types and Operators)	$lecture_4$.txt — FoCS Lecture 4 (6 октября 2026): review, variables and data types, operators epilogue · $Week_4$.txt — лаборатория «Programming in C — Data Types and Operators» (глава 5 учебника)	—
5	Управляющие конструкции (Control Structures)	$Lecture_5$.txt — Lecture 5 (13 октября 2026): блоки, условия, циклы, функции, области видимости · Week-5-Control-Structures.txt — лаборатория «Chapter 6 · Control structures»	bug_{huntc}.txt (задачи 1–4: ветвления) · $bug_{huntloops}$.txt

Неделя	Тема	Лекции/лаборатория	Доп. файлы
6	Функции и структуры (Functions and Structures)	Lecture$_{6}$.txt — TashTech Lecture 6 (20 октября 2026): control flow, I/O, строки, файлы, командная строка · Week-6-Functions-and-Structures.txt — лаборатория «WEEK 6 · CHAPTER 7»: от блоков Scratch к кофемашине	bug$_{huntc}$.txt (задачи 5–6: функции) · проект «кофейный автомат» (Stock, Recipe, Cup)
7	Указатели, строки и рекурсия (Pointers, Strings, Recursion)	Lecture$_{71}$.txt — TashTech Lecture 7 (27 октября 2026): указатели и адреса, массивы и адресная арифметика, строки, поиск и сортировки · Recursion-Lecture.txt — «WEEK 7 · CHAPTER 8: Recursion»	bug$_{huntpointersstrings}$.txt · bug$_{huntpointersstringsanswerkey}$.txt (ключ инструктора)

Недели 4–7: темы и файлы курса

2. Неделя 4 — Типы данных и операторы (Data Types and Operators)

Файлы недели: Week$_{4}$.txt (лаборатория «Programming in C — Data Types and Operators», глава 5) · lecture$_{4}$.txt (FoCS Lecture 4 — October 6, 2026)

Неделя 4 — фундамент всего курса: переменные и типы данных C, их размеры и точность, целочисленное и вещественное деление, арифметические, логические, побитовые операторы, инкремент/декремент, компактное присваивание, тернарный оператор, преобразования типов и приоритет операторов. Лаборатория недели: точность сферы (sphere$_{precision}$), починка правила високосного года (leap$_{year}$), мост в память (байты и массивы символов) и алфавитное склеивание (alphabetical$_{join}$); в отдельной сессии LMS — задания Circle precision и Character processor, а пятибуквенное склеивание — challenge. Ниже — все правила, все вопросы, тесты и термины недели.

2.1. Ключевые положения («все правила» недели 4)

Типы данных C: объявление, имена переменных, константы

C — быстрый, маленький, универсальный, платформенно-независимый язык; используется для системного программирования (компиляторы и интерпретаторы, операционные системы, СУБД, микроконтроллеры). C — статический (компилируемый), типизированный, структурированный и императивный. «C is quirky, flawed, and an enormous success» — Ритчи. **Тип данных** объекта в памяти определяет множество значений, которые он может принимать, и операции, которые к нему применимы; C — слабо типизированный язык: допускает неявные преобразования и принудительное (потенциально опасное) приведение типов (casting).

Семейство типов C невелико: числовые (int, float, double), символьные (char), определяемые пользователем (struct, union). Базовая таблица лаборатории:

Тип	Объявление C	Типичное использование
int	int score = 72;	целые числа
char	char grade = 'B';	один базовый символ
float	float temperature = 23.5f;	приближённые вещественные значения

Тип	Объявление C	Типичное использование
double	double radius = 3.0;	приближённые вещественные значения
bool	bool passed = true;	true или false
string	string name = "Ada";	текст (через CS50)

Переменные и типы [Источник: Week$_4$, слайд 2]

Уточнения слайда: в C11 bool — макрос из <stdbool.h>; string — CS50-typedef для char * (хранит адрес, а не сам текст); один char — один байт, «базовый символ» вроде B, а не любой Unicode-символ; переменную типа void объявить нельзя. **int main(void)**: int — тип возвращаемого значения, void в скобках — «нет параметров»; **void example(void)**: первый void — «нет возвращаемого значения», второй — «нет параметров».

Общий формат объявления: **type variable-name [= value]**. Примеры лекции:

```
char x;                /* uninitialized */
char x = 'A';          /* initialized to 'A' */
char x = 'A', y = 'B'; /* multiple variables initialized */
char x = y = 'Z';      /* multiple initializations */
```

[Источник: lecture$_4$, слайд 12]

Правила имён переменных: имена могут содержать буквы, цифры и _; должны начинаться с буквы; ключевые слова (for, while и т.д.) использовать нельзя; имена чувствительны к регистру — int x; и int X; объявляют две разные переменные. Pop quiz лекции: int money$owed; — неверно (символ $); int total$_{count}$; — верно; int score2; — верно; int 2ndscore; — неверно (начинается с цифры); int long; — неверно (ключевое слово).

Константы — литеральные/фиксированные значения. Целые: int i = 3; long l = 3; unsigned long ul = 3UL; шестнадцатеричное int i = 0xA;; восьмеричное int i = 012;. Вещественные: float pi = 3.14159; float pi = 3.141F; double pi = 3.14159265358979323 84L;. Символьные: 'A', '\x41' (hex), '\0101' (octal). Строковые: "hello world"; "hello" "world" — то же самое, что "hello world". Перечисления: enum BOOL {NO, YES} → NO = 0, YES = 1; enum COLOR {R = 1, G, B, Y = 10} → G = 2, B = 3 — энумератор без явного значения продолжает нумерацию от предыдущего.

[Источник: lecture$_4$, слайды 10–11; Week$_4$, слайд 2]

Размеры, диапазоны и точность: sizeof, float и double

Отдельные размеры зависят от машины/компилятора, но гарантировано: sizeof(char) < sizeof(short) <= sizeof(int) <= sizeof(long) и sizeof(char) < sizeof(short) <= sizeof(float) <= sizeof(double). Беззнаковая версия имеет примерно вдвое больший диапазон, чем знаковая; знаковый и беззнаковый char различаются только в арифметических выражениях. Для многобайтовых чисел важен порядок байтов: **big endian** — старшие биты (MSBs) по младшему адресу (процессоры powerpc; сетевой порядок — network order); **little endian** — младшие биты (LSBs) по младшему адресу (все x86-совместимые процессоры). Это «проблема NUXI».

Тип	signed	unsigned
short	short int x; short y;	unsigned short x; unsigned short int y;
по умолчанию	int x;	unsigned int x;
long	long x;	unsigned long x;
float	float x;	N/A
double	double x;	N/A

Тип	signed	unsigned
char	char x; signed char x;	unsigned char x;

Числовые типы: signed и unsigned [Источник: lecture$_4$, слайд 7]

Таблицы лаборатории (измерено на macOS arm64, 8 бит на байте; на других платформах значения могут отличаться):

Тип	Бит	Диапазон
char	8	–128 … 127
signed char	8	–128 … 127
unsigned char	8	0 … 255
short	16	–32 768 … 32 767
unsigned short	16	0 … 65 535
int	32	–2 147 483 648 … 2 147 483 647
unsigned int	32	0 … 4 294 967 295
long	64	–9 223 372 036 854 775 808 … 9 223 372 036 854 775 807
unsigned long	64	0 … 18 446 744 073 709 551 615
long long	64	–9 223 372 036 854 775 808 … 9 223 372 036 854 775 807
unsigned long long	64	0 … 18 446 744 073 709 551 615

Целые типы: размеры и диапазоны; signed int = int, short int = short, long int = long [Источник: Week$_4$, слайды 27–28]

Тип	Бит	Десятичная точность
float	32	6 значащих цифр
double	64	15 значащих цифр
long double	64	15 значащих цифр (как double на этой платформе)

Вещественные типы: точность из FLT_{DIG}, DBL_{DIG}, $LDBL_{DIG}$ (<float.h>) [Источник: Week$_4$, слайд 29]

Выводы лаборатории: у **float** — 6, у **double** — 15 значащих десятичных цифр (значащие цифры, а не знаки после запятой); для радиуса и объёма сферы берём **double**. Задача сферы:

$$V = (4 / 3) \times \pi \times r^3$$

```
const double pi = 3.1415;
double radius = get_double("Radius: ");
// Today's input range: 0 < radius <= 1000
```

[Источник: Week$_4$, слайд 4]

Деление и остаток: главная ловушка недели

Правило лекции: **типы операндов, а не тип приёмника, решают, как работает деление**. float x = 3/2; даёт x = 1 (целое деление); float x = 3.0/2; даёт x = 1.5 (вещественное деление); int x = 3.0/2; снова даёт 1 (преобразование к int). Остаток: int x = 3%2; → 1; int y = 7; int x = y%4; → 3; int x = y%10; → 7.

Деление выполняется **до** присваивания — лабораторная таблица предсказаний:

Выражение	Сохранено в double	Напечатано с %f
4 / 3	1.0	1.000000
4.0 / 3.0	≈ 1.333333…	1.333333

Выражение	Сохранено в double	Напечатано с %f
(double)(4 / 3)	1.0	1.000000

Типы операндов решают, как работает / [Источник: Week$_4$, слайд 6]

```
double fraction = 4 / 3;
printf("%f\n", fraction);
```

Prediction 1 лаборатории: этот фрагмент печатает **1.000000** (вариант A) — целочисленное деление даёт 1, и лишь затем присваивание преобразует 1 в double. Меняет ли тип приёмника выполнение 4/3? Нет.

Форматирование не улучшает хранимое значение: при radius = 3 один и тот же правильный объём печатается как **113.1** (%.1f) и **113.0940** (%.4f) — printf меняет только представление, а не сохранённое число. Допустимый радиус: строго больше 0 и не больше 1000; ноль, отрицательные значения и всё свыше 1000 программа отвергает (проверка положительного диапазона отсекает и нечисловой ввод).

[Источник: Week$_4$, слайды 3–8; lecture$_4$, слайд 15]

Операторы сравнения и логические операторы: короткое замыкание

Операторы сравнения сопоставляют два операнда и дают «булев» результат: в C **любое ненулевое значение (по соглашению 1) — истина, 0 — ложь**. Примеры лекции: 3 > 2 → 1; 2.99 > 3 → 0; 3 >= 3 → 1; 3 < 3 → 0; 'A' < 'B' → 1; 3 == 3 → 1; 'A' == 'a' → 0; 3 != 3 → 0; 2.99 != 3 → 1. Ловушки: **== (равенство) — не = (присваивание)**; сравнение == вещественных переменных ненадёжно из-за конечной точности.

Логические операторы && (И), || (ИЛИ), ! (НЕ):

```
((9/3) == 3) && (2*3 == 6); /* evaluates to 1 */
('A' == 'a') && (3 == 3)    /* evaluates to 0 */
2 == 3 || 'A' == 'A';       /* evaluates to 1 */
2.99 >= 3 || 0              /* evaluates to 0 */
!(3 == 3);                  /* evaluates to 0 */
!(2.99 >= 3)                /* evaluates to 1 */
```

Короткое замыкание (short circuit): вычисление выражения прекращается, если значение условия уже определено; следите за побочными эффектами. Примеры лекции: (3 == 3) || ((c = getchar()) == 'y') — вторая часть НЕ вычисляется; (0) && ((x = x + 1) > 0) — вторая часть НЕ вычисляется, x не изменится. Лабораторная таблица (граница массива перед доступом к символу):

i	i < 3	Читать символы?
0, 1, 2	true	да
3	false	нет

i < LENGTH && first[i] == second[i]: && пропускает правую часть, когда левая ложна [Источник: Week$_4$, слайд 22]

[Источник: lecture$_4$, слайды 16–18; Week$_4$, слайд 22]

Инкремент/декремент, компактное присваивание и тернарный оператор

Постфикс: x++ — короткая запись x = x+1; y = x++ эквивалентно y = x; x = x+1; — x вычисляется **до** инкремента (аналогично x−−). **Префикс**: ++x; y = ++x эквивалентно x = x+1; y = x; — x вычисляется **после** инкремента. Компактные операторы присваивания для шаблона var = var (op) expr:

```
x += 1   /* is the same as x = x+1 */
x -= 1   /* is the same as x = x-1 */
x *= 10  /* is the same as x = x*10 */
x /= 2   /* is the same as x = x/2 */
x %= 2   /* is the same as x = x%2 */
```

Тернарный оператор ?: — синтаксический сахар для шаблона if (cond) x = <expra>; else x = <exprb>;:

```
sign = x > 0 ? 1 : -1;
/* if (x > 0)
       sign = 1;
   else
       sign = -1; */

isodd = x % 2 == 1 ? 1 : 0;
/* if (x % 2 == 1)
       isodd = 1;
   else
       isodd = 0; */
```

[Источник: lecture$_4$, слайды 19–23]

Побитовые операторы

Оператор	Смысл	Примеры
&	AND	0x77 & 0x3 → 0x3; 0x77 & 0x0 → 0
|	OR	0x700 | 0x33 → 0x733; 0x070 | 0 → 0x070
^	XOR	0x770 ^ 0x773 → 0x3; 0x33 ^ 0x33 → 0
<<	сдвиг влево	0x01 << 4 → 0x10; 1 << 2 → 4
>>	сдвиг вправо	0x010 >> 4 → 0x01; 4 >> 1 → 2

Побитовые операторы [Источник: lecture$_4$, слайд 21]

Правила слайда: AND истинен, только если истинны оба операнда; OR — если хотя бы один; XOR — если ровно один.

Преобразования типов и приоритет операторов

При повышении до большей точности данные сохраняются — компилятор делает это автоматически для выражений со смешанными типами; правило (с исключениями): каждый член бинарного выражения повышается до точности операнда наивысшей точности. Автоматическое преобразование char → int позволяет сравнивать и «вычислять» символы:

```
int i;
float f;
f = i + 3.14159; /* i is promoted to float,
                    f = (float)i + 3.14159 */

isupper = (c >= 'A' && c <= 'Z') ? 1 : 0;
/* c and literal constants are converted to int */
if (!isupper)
    c = c - 'a' + 'A';
/* subtraction is possible because of integer conversion */
```

Приоритет и порядок вычисления: ++, −−, (cast), sizeof — наивысший приоритет; *, /, % выше, чем +, −; ==, != выше, чем &&, ||; операторы присваивания — очень низкий. Используйте скобки ( ) щедро, чтобы избежать двусмысленностей и побочных эффектов приоритета:

```
y = x*3+2            /* same as y = (x*3)+2 */
x != 0 && y == 0     /* same as (x != 0) && (y == 0) */
d = c >= '0' && c <= '9'  /* same as d = (c >= '0') && (c <= '9') */
```

[Источник: lecture$_4$, слайды 24–25]

Лаборатория: високосный год и массивы символов

Високосный год. Обычный год — 365 дней (в феврале 28), високосный — 366 (29 февраля): дополнительный день держит календарь в согласии с временами года. Правило читается сверху вниз: кратен 400? → високосный; иначе кратен 100? → обычный; иначе кратен 4? → високосный; иначе обычный. 2000 — високосный (кратен 400); 1900 — обычный (кратен 100, но не 400); 2024 — високосный; 2025 — обычный; 2100 — обычный (exit check: кратен 100, но не 400). Компактное выражение лаборатории:

```
year % 400 == 0 || (year % 4 == 0 && year % 100 != 0)
```

Повторяемый ввод («Go back to line 1» — псевдокод повторения в стиле CS50): читать год до цикла и снова в конце цикла while; год 0 или отрицательный завершает без классификации; больше 9999 — сообщение об ошибке и останов. Если забыть повторное чтение — положительный год никогда не изменится, цикл повторится навсегда. Первое правило year % 4 == 0 классифицирует 2024 и 2025 правильно — эти два теста не вскрывают баг столетий; проверяйте 1900 и 2000.

Мост в память. Модуль RAM → один чип → концептуальная сетка: каждый квадрат — один байт; sizeof(char) — 1 байт C, число битов даёт CHAR_{BIT} (на учебной машине 8); измеренный размер int — 4 байта. Массивы из трёх символов:

```
char first[3]  = {'d', 'o', 'g'};
char second[3] = {'c', 'a', 't'};
```

Index	0	1	2
first	d	o	g
second	c	a	t

Индексы 0–2 · нет терминатора '\0' · это не C-строки [Источник: Week_4, слайд 18]

Алфавитное склеивание (dog + cat → catdog): сравниваем два массива целиком по первому различающемуся символу (car/cat различаются на индексе 2: r и t), затем копируем «меньший» массив первым. Скан равного префикса: i < LENGTH && first[i] == second[i] — граница обязана стоять до доступа к символу; равные массивы доходят до границы: $\text{first}_{\text{earlier}}$ = i == LENGTH || first[i] < second[i]. Копирование: i заполняет joined[0..2], LENGTH + i — joined[3..5]:

Index	0	1	2	3	4	5
joined	c	a	t	d	o	g

Смещение копирования: i заполняет 0–2, LENGTH + i заполняет 3–5 [Источник: Week_4, слайд 23]

Печать: в joined нет финального '\0', поэтому %s непригоден — печатаем ровно шесть символов через **%c**.

[Источник: Week_4, слайды 9–26]

2.2. Savollar недели 4 — вопросы лекции и лаборатории с модельными ответами

S1. Разогрев лаборатории: какие типы хранят целые числа, десятичные значения и один символ?

Ответ. int — для целых чисел; float или double — для десятичных (вещественных) значений; char — для одного символа. Выполнение начинается с main; stdio.h объявляет printf, cs50.h — $\text{get}_{\text{double}}$.

S2. Что означают void в int main(void) и в void example(void)?

Ответ. В int main(void) int — тип возвращаемого значения, а void в скобках означает «нет параметров». В void example(void) первый void — «нет возвращаемого значения», второй — «нет параметров».

Обычную переменную типа void объявить нельзя.

S3. Почему для радиуса и объёма сферы выбран double, а не float?

Ответ. Радиус может иметь дробную часть, а на нашей платформе double точнее float: 15 значащих десятичных цифр против 6 (DBL_{DIG} против FLT_{DIG}). При этом double не делает приближение pi = 3.1415 точным — вещественные значения остаются приближёнными.

S4. Что напечатает double fraction = 4 / 3; printf("%f\n", fraction);?

Ответ. 1.000000. Деление происходит до присваивания: оба операнда целые, 4/3 даёт 1, и только затем 1 преобразуется в double. Тип приёмника не меняет то, как выполняется деление, — решают типы операндов. Так же (double)(4/3) даёт 1.0, а 4.0/3.0 — 1.333333.

S5. Программа сферы печатает один и тот же объём при radius = 3 через %.1f и %.4f. Улучшают ли лишние знаки хранимое значение?

Ответ. Нет. Выводы — 113.1 и 113.0940: это одно и то же сохранённое число; форматирование меняет только представление (printf precision controls the display, not the stored value).

S6. Pop quiz II лекции: int x = 017; int y = 12; — x больше y?

Ответ. Да. 017 — восьмеричный литерал (как int i = 012; из таблицы констант — octal number): $017_8 = 1 \cdot 8 + 7 = 15$, а $15 > 12$.

S7. Чему равно XL в enum sz {S = 0, L = 3, XL}; и в enum sz {S = 0, L = −3, XL};?

Ответ. Энумератор без явного значения продолжает нумерацию от предыдущего — как в enum COLOR {R = 1, G, B, Y = 10}, где G = 2, B = 3. Поэтому в первом случае XL = 4, во втором XL = −2.

S8. Почему 'A' < 'B' даёт 1, а 'A' == 'a' даёт 0?

Ответ. char автоматически преобразуется в int (код символа). Коды 'A' и 'B' идут по возрастанию, поэтому сравнение кодов даёт 1; коды 'A' и 'a' различны, поэтому равенство ложно. Из-за этого же преобразования возможна арифметика c = c - 'a' + 'A'; (перевод строчной буквы в заглавную).

S9. Что такое короткое замыкание и что произойдёт с x в (0) && ((x = x + 1) > 0)?

Ответ. Вычисление выражения прекращается, если результат уже определён. Левый операнд && равен 0, значит всё выражение ложно, и правая часть — с побочным эффектом x = x + 1 — не вычисляется вовсе: x не изменится. Аналогично в (3 == 3) || ((c = getchar()) == 'y') getchar не вызовется. В цикле i < LENGTH && first[i] == second[i] при i = 3 обращение к first[3] не происходит.

S10. В чём разница между y = x++ и y = ++x?

Ответ. Постфикс: y = x++; эквивалентно y = x; x = x + 1; — x вычисляется до инкремента. Префикс: y = ++x; эквивалентно x = x + 1; y = x; — x вычисляется после инкремента. Для декремента — так же: y = x−− и y = −−x.

S11. Как работает тернарный оператор в sign = x > 0 ? 1 : −1;?

Ответ. Это синтаксический сахар для if (x > 0) sign = 1; else sign = −1;: условие стоит перед ?, значение при истине — между ? и :, при лжи — после :. Код становится короче и понятнее; второй пример слайда — isodd = x % 2 == 1 ? 1 : 0;.

S12. Какие входные данные должна отвергать программа объёма сферы?

Ответ. Ноль, отрицательные радиусы и значения больше 1000: допустимый диапазон 0 < radius <= 1000. Проверка положительного диапазона также отвергает нечисловой (nonfinite) ввод.

S13. Почему 1900 — обычный год, а 2000 — високосный? Почему 2100 — обычный?

Ответ. 2000 кратен 400 — високосный. 1900 кратен 100, но не 400 — обычный; 2100 — так же (кратен 100, но не 400), а правило проверяет кратность 400 раньше кратности 100. Компактное условие: year % 400 == 0 || (year % 4 == 0 && year % 100 != 0); 2024 по нему високосный, 2025 — обычный.

S14. Почему массив char first[3] = {'d', 'o', 'g'} нельзя напечатать %s и при чём здесь порядок в &&?

Ответ. В массиве ровно три символа и нет терминатора '\0' — это не C-строка: %s продолжал бы читать память за массивом до случайного нулевого байта. Печатают ровно три символа через %c. А в цикле сравнения граница обязана идти первой: i < LENGTH && first[i] == second[i] — при i = 3 левая часть ложна, && пропускает правую, и выхода за границы массива не происходит.

2.3. Testlar недели 4

T1. Какой тип C хранит один базовый символ?

a) int
b) char
c) double
d) bool

Ответ: b) char

Пояснение: char grade = 'B'; — one basic character; int — целые числа, double — приближённые вещественные, bool — true/false.

T2. Сколько значащих десятичных цифр у double на платформе курса?

a) 6
b) 8
c) 15
d) 32

Ответ: c) 15

Пояснение: DBL_{DIG} = 15 (у float — 6, FLT_{DIG}). Это значащие цифры, а не число знаков после запятой; long double на этой платформе тоже 15.

T3. Что напечатает double fraction = 4 / 3; printf("%f\n", fraction);?

a) 1.000000
b) 1.333333
c) ошибку компиляции
d) 0.000000

Ответ: a) 1.000000

Пояснение: 4/3 — целочисленное деление, даёт 1; преобразование в double происходит при присваивании. Правильный ответ prediction 1 лаборатории — вариант A.

T4. Какая запись означает целое число 15?

a) int x = 0xA;
b) int x = 017;
c) int x = 15.0;
d) int x = '15';

Ответ: b) int x = 017;

Пояснение: 017 — восьмеричный литерал: 1·8 + 7 = 15. 0xA — шестнадцатеричное 10; '15' — не символ (в одинарных кавычках допустим только один символ).

T5. Какое объявление переменной корректно?

a) int money$owed;

b) int 2ndscore;

c) int total_count;

d) int long;

Ответ: c) int total_count;

Пояснение: pop quiz лекции: $ недопустим; имя не может начинаться с цифры; long — ключевое слово. Буквы, цифры и _ — можно, но первой должна быть буква.

T6. Чему равно y % 10 при int y = 7;?

a) 0

b) 3

c) 7

d) 1

Ответ: c) 7

Пояснение: int y = 7; int x = y % 10; / produces 7 / — 7 меньше 10, остаток равен самому числу; а y % 4 даёт 3.

T7. Чему равно 0x01 << 4?

a) 0x10

b) 0x01

c) 4

d) 0x14

Ответ: a) 0x10

Пояснение: сдвиг влево на 4 бита: 0x01 << 4 evaluates to 0x10; аналогично 1 << 2 → 4, 0x010 >> 4 → 0x01, 4 >> 1 → 2.

T8. Какая группа операторов имеет наивысший приоритет?

a) = и +=

b) && и ||

c) * и /

d) ++, −−, (cast), sizeof

Ответ: d) ++, −−, (cast), sizeof

*Пояснение: ++, −−, (cast), sizeof — наивысший приоритет; затем *, /, %; затем +, −; ==, != выше &&, ||; присваивание — очень низкий приоритет.*

T9. Выражение y = x*3 + 2 эквивалентно…

a) y = x*(3+2)

b) y = (x*3)+2

c) y = x+(3*2)

d) y = 3*(x+2)

Ответ: b) y = (x*3)+2

Пояснение: , /, % имеют приоритет выше, чем + и −, поэтому умножение выполняется первым — как в примере слайда «same as y = (x3)+2».

T10. Что произойдёт с x в выражении (0) && ((x = x + 1) > 0)?

a) x увеличится на 1

b) x не изменится

c) ошибка компиляции

d) x станет равным 0

Ответ: b) x не изменится

Пояснение: короткое замыкание: левый операнд && уже ложен, правая часть с побочным эффектом не вычисляется — «be careful of any side effects».

2.4. Atamalar — глоссарий недели 4

Термин (atama)	Значение
Тип данных (datatype)	определяет множество значений объекта в памяти и допустимые операции над ним; C — слабо типизированный язык с неявными преобразованиями и приведением (casting)
Переменная (variable)	именованное связывание/ссылка на значение, хранящееся в памяти, или вычислимое выражение (int x = 0, y = 0; y = x + 2;)
Выражение (expression)	комбинация значений, переменных, операторов и функций (y = x + 2)
Оператор (operator)	задаёт манипуляцию объектом; унарный (−, ++), бинарный (+, −, *, /), тернарный (?:)
Целочисленное деление	деление двух целых: дробная часть отбрасывается (4/3 → 1, 3/2 → 1); тип приёмника не влияет — важны типы операндов
Остаток (modulus, %)	остаток от деления целых: 3 % 2 → 1, 7 % 4 → 3, 7 % 10 → 7
Значащие цифры	точность вещественного типа: float — 6, double — 15 значащих десятичных цифр (FLT_{DIG}, DBL_{DIG} из <float.h>)
Порядок байтов (endianness)	big endian — старшие байты по младшему адресу (powerpc, network order); little endian — младшие по младшему (x86); «проблема NUXI»
Константа (constant)	литеральное/фиксированное значение: 3, 3UL, 0xA (hex), 012 (octal), 'A', '\x41', "hello world"
Перечисление (enum)	именованные целые константы; без явного значения нумерация продолжается: enum COLOR {R=1,G,B,Y=10} → G=2, B=3
Короткое замыкание (short circuit)	прекращение вычисления && и ||, когда результат уже определён; побочные эффекты правой части могут не выполниться
Приоритет операторов (precedence)	порядок: ++/−−/(cast)/sizeof → *,/,% → +,− → ==,!= → &&,|| → присваивание; скобки ( ) — щедро
Инкремент/декремент	x++ (постфикс: значение до увеличения) и ++x (префикс: значение после увеличения)
Високосный год (leap year)	год, кратный 400, либо кратный 4 и не кратный 100: year % 400 == 0 || (year % 4 == 0 && year % 100 != 0)

3. Неделя 5 — Управляющие конструкции (Control Structures)

Файлы недели: Week-5-Control-Structures.txt (лаборатория «Chapter 6 · Control structures») · $Lecture_5$.txt (Lecture 5 — October 13, 2026)

Неделя 5 учит программу «принимать решения и повторять задачи»: блоки и составные операторы, условия (if / else if / else, switch), циклы (while, for, do-while), break и continue, а в лекции — ещё функции, модульное программирование и области видимости (static, register, extern). Лаборатория проходит по главе 6 учебника: пример Melody (мелодия из имени), подсчёт длины строки в стиле CS50, необязательные фрагменты do-while и continue. Ниже — все правила, вопросы, тесты и термины недели.

3.1. Ключевые положения («все правила» недели 5)

Блоки и составные операторы

Простой оператор заканчивается точкой с запятой: z = foo(x+y);. Несколько операторов объединяются в составной оператор (блок) фигурными скобками. Блок может стоять там, где ожидается простой оператор; компилируется как единое целое; внутри можно объявлять переменные; блок может быть пустым {}; после закрывающей скобки точка с запятой не ставится. Блоки можно вкладывать друг в друга:

```
{
    int temp = x + y;
    z = foo(temp);
    {
        float temp2 = x * y;
        z += bar(temp2);
    }
}
```

[Источник: Lecture$_5$, слайды 4–6]

Условие в C и семейство if / else if / else

В отличие от C++ или Java, в C89/C90 нет логического типа; в C99 bool доступен через <stdbool.h>. **Условие — это выражение (или серия выражений)**, например n < 3 или x < y || z < y. Выражение ненулевое ⇒ условие истинно; выражение должно быть числовым (или указателем):

```
const char str[] = "some text";
if (str) /* string is not null */
    return 0;
```

if: вычислить условие; если истинно — выполнить внутренний оператор; иначе — ничего. **else** (необязателен): выполнить оператор, если условие ложно. **else if**: дополнительные альтернативные пути; условия проверяются по порядку до первого истинного — если истинны несколько, выполнится только первый; эквивалентно вложенным if. Любой внутренний оператор может быть блоком:

```
if (x % 2) y += x / 2;

if (x % 2 == 0)
    y += x/2;
else
    y += (x+1)/2;

if (x % 2 == 0) y += x / 2;
else if (x % 4 == 1) y += 2 * ((x + 3) / 4);
else
    y += (x + 1) / 2;
```

[Источник: Lecture$_5$, слайды 7–11]

Вложенные if — классический подвох: else без скобок связывается с **ближайшим внутренним** if. Чтобы привязать else к внешнему if — фигурные скобки:

```
if (x % 4 == 0)
```

```
    if (x % 2 == 0) y = 2;
    else
        y = 1;
/* else принадлежит внутреннему if! */

if (x % 4 == 0) {
    if (x % 2 == 0) y = 2;
} else
    y = 1;
/* теперь else связан с внешним if */
```

[Источник: Lecture$_5$, слайды 12–13]

switch: case, break, default и fall-through

switch — альтернативный условный оператор: на входе целочисленная (или символьная) переменная; рассматриваются случаи (case) для её значения. Содержимое switch — блок; **case-метки — разные точки входа в блок**; компилятор сравнивает переменную с каждой меткой по порядку; при совпадении выполняет внутренний код **до ближайшего break**; без break выполнение «проваливается» (falls through) в следующий case без повторной проверки; default — ветка «иначе». C-принципы лаборатории: switch принимает целочисленное выражение (включая char); case-метки — различные целые константные выражения; для диапазонов и составных условий используйте if. Полный пример лаборатории (grade):

```
if (grade == 'A') {
    printf("Perfect!\n");
} else if (grade == 'B') {
    printf("You did good!\n");
} else if (grade == 'C') {
    printf("You did okay!\n");
} else if (grade == 'D') {
    printf("At least it's not an F!\n");
} else if (grade == 'F') {
    printf("YOU FAILED!\n");
} else {
    printf("That's not a valid grade!\n");
}
```

```
switch (grade) {
case 'A':
    printf("Perfect!\n");
    break;
case 'B':
    printf("You did good!\n");
    break;
case 'C':
    printf("You did okay!\n");
    break;
case 'D':
    printf("At least it's not an F!\n");
    break;
case 'F':
    printf("YOU FAILED!\n");
    break;
default:
    printf("That's not a valid grade!\n");
}
```

Каждая проверка равенства из лестницы if/else-if превращается в case-метку, else — в default; break выходит из switch (выполнение продолжается после закрывающей скобки — break не завершает

программу). Обе программы дают одинаковые сообщения для всех оценок A/B/C/D/F и W. Множественные case-метки ставятся друг за другом:

```
switch (ch) {
case 'Y':  /* ch == 'Y' */
    /* do something */
    break;
case 'N':  /* ch == 'N' */
    /* do something else */
    break;
default:   /* otherwise */
    /* do a third thing */
    break;
}
```

```
switch (ch) {
case 'Y':
case 'y':
    /* do something if ch == 'Y' or ch == 'y' */
    break;
}
```

[Источник: Week-5-Control-Structures, слайды 5–6; Lecture$_5$, слайды 14–15]

Пример 6.1: мелодия из имени

Book Example 6.1 читает по одному символу, вычисляет ASCII-код по модулю 7 и печатает имя ноты (звук не воспроизводится). Для 'A': код 65; 65 % 7 = 2; ожидаемая нота — C. В учебной версии 01$_{\text{melodybuggy}}$.c после case 2 **пропущен break** — это намеренная ошибка для разбора: ввод A# печатает C, затем D (проваливание в case 3 без новой проверки). Разбор в debug50: точка останова на printf case 2, Step Over в case 3; починка — добавить break после printf в case 2; проверка на B# и немедленном #. Исправленная версия читает символ, проверяет успешный ввод и сторожевой символ # (scanf(" %c", &C) пропускает пробельные символы), выбирает ноту и читает снова; EOF и # завершают цикл; break внутри case оставляет switch, поэтому происходит новое чтение. Ввод AB# даёт ноту C, затем ноту D.

Символ	ASCII-код	65 % 7	Ожидаемая нота
A	65	2	C

Мелодия: 'A' → код 65 → остаток 2 → нота C [Источник: Week-5-Control-Structures, слайд 7]

Циклы: while, for, do-while

while — простейший цикл: вычислять тело, пока условие истинно. Условие вычисляется **до** тела, поэтому тело может не выполниться ни разу:

```
while (/* condition */)
    /* loop body */
```

for — «счётный» цикл: в скобках три выражения через точку с запятой — инициализация (i = 1), условие (i <= n), инкремент (i++). Выражения могут быть пустыми (пустое условие считается истинным). Порядок выполнения: инициализация один раз; условие перед каждым телом; тело; обновление; снова условие. Эквивалент while и вариант с оператором «запятая» (запятая — оператор с низшим приоритетом, вычисляется слева направо; это не то же, что запятая между аргументами функции):

```
int factorial(int n) {
    int i, j = 1;
    for (i = 1; i <= n; i++) j *= i;
    return j;
}
```

```
int factorial(int n) {
    int j = 1;
    int i = 1; /* initialization */
    while (i <= n /* condition */) {
        j *= i;
        i++; /* increment */
    }
    return j;
}
```

```
int factorial(int n) {
    int i, j;
    for (i = 1, j = 1; i <= n; j *= i, i++)
        ;
    return j;
}
```

Сравнение из лаборатории (§6.2.1): оба фрагмента печатают **1 2 3**; вариант while объявляет счётчик int i = 1; перед циклом, вариант for — в заголовке, и этот счётчик виден только внутри цикла (не ссылайтесь на него после):

```
int i = 1;
while (i <= 3)
{
    printf("%d ", i);
    i++;
}
```

```
for (int i = 1; i <= 3; i++)
{
    printf("%d ", i);
}
```

do-while отличается от while тем, что условие вычисляется **после** каждой итерации: тело выполняется хотя бы один раз; обратите внимание на точку с запятой в конце:

```
char c;
do {
    /* loop body */
    puts("Keep going? (y/n) ");
    c = getchar();
    /* other processing */
} while (c == 'y' /* other conditions */);
```

[Источник: Lecture$_5$, слайды 18–22; Week-5-Control-Structures, слайды 10–11]

break и continue

Иногда нужно завершить цикл досрочно. **break;** выходит из самого внутреннего цикла или оператора switch:

```
char c;
do {
    /* loop body */
    puts("Keep going? (y/n) ");
    c = getchar();
    if (c != 'y')
        break;
    /* other processing */
} while (/* other conditions */);
```

continue; пропускает остаток тела самого внутреннего цикла и переходит к его условию. Пример лекции — НОД (gcd) с макросом min: если i не делит a, переходим к следующему i; если i делит и a, и b — запоминаем:

```
#define min(a,b) ((a)<(b)?(a):(b))

int gcd(int a, int b) {
    int i, ret = 1, minval = min(a, b);
    for (i = 2; i <= minval; i++) {
        if (a % i) /* i not divisor of a */
            continue;
        if (b % i == 0) /* i is divisor of both a and b */
            ret = i;
    }
    return ret;
}
```

[Источник: Lecture$_5$, слайды 23–24]

Функции, области видимости и модули (по лекции 5)

Лекция 5 выходит за пределы лаборатории и знакомит с функциями: **divide and conquer** — разбить программу на более простые подзадачи. Пример — линейное диофантово уравнение ax + by = c: получить a, b, c; вычислить g = gcd(a, b); если c не кратно gcd — решений нет; запустить расширенный алгоритм Евклида; промасштабировать x и y на (c/g); напечатать решение. Алгоритм Евклида опирается на gcd(a, b) = gcd(b, a mod b):

```
int gcd(int a, int b) {
    while (b) { /* if a < b, performs swap */
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}
```

Функции возвращают не более одного значения: для нескольких выходов (расширенный Евклид возвращает gcd, x и y) лекция использует глобальные переменные — переменные, объявленные вне функции: существуют всё время жизни программы, доступны из любой функции. **Модуль** в C: интерфейс (заголовочный файл .h) + реализация (вспомогательные исходные/объектные файлы). Заголовок euclid.h содержит прототипы функций и extern-объявления глобальных переменных; extern сообщает компилятору, что переменная определена в другом файле; компиляция одной командой gcc -g -O0 -Wall diophant.c euclid.c -o diophant.o (нужно компилировать и линковать оба файла):

```
/* ensure included only once */
#ifndef EUCLID_H
#define EUCLID_H
/* global variables (declared in euclid.c) */
extern int x, y;
/* compute gcd */
int gcd(int a, int b);
/* compute g = gcd(a, b) and solve ax+by=g */
int ext_euclid(int a, int b);
#endif
```

Область видимости (scope) — регион, в котором переменная действительна; обычно это блок с объявлением переменной. Переменные вне функций — глобальные. Пример лекции: глобальный int nmax = 20; и цикл Фибоначчи в main; если внутри main объявить локальный int nmax = 25;, он **затеняет** глобальный — цикл дойдёт до 25 и строк будет напечатано 25. **static**: вне функции —

переменная/функция видна только внутри файла (нельзя extern); внутри функции — переменная локальна, но инициализируется один раз при запуске программы и не переинициализируется при вызовах (static int somePersistentVar = 0;). **register**: явный запрос хранить переменную в регистре; только локальные переменные и аргументы; лишние/недопустимые объявления игнорируются; у register-переменной нельзя взять адрес — указатель на неё недопустим.

```
int nmax = 20;
/* The main() function */
int main(int argc, char **argv) /* entry point */
{
    int a = 0, b = 1, c, n;
    printf("%3d: %d\n", 1, a);
    printf("%3d: %d\n", 2, b);
    for (n = 3; n <= nmax; n++) {
        c = a + b;
        a = b;
        b = c;
        printf("%3d: %d\n", n, c);
    }
    return 0; /* success */
}
```

[Источник: Lecture$_5$, слайды 25–43]

Лаборатория: длина строки, do-while и continue

Длина строки (CS50-дополнение): string name = get$_{string}$("Name: "); строка хранит H, I, ! и терминатор '\0'. Алгоритм: начать счёт с нуля; считать каждый символ; остановиться на конечной метке; **терминатор не считается**. Для HI!: n = 0 читает H, n = 1 читает I, n = 2 читает !, n = 3 достигает NUL — условие ложно до очередного приращения, печатается 3. Пустая строка даёт 0. Без strlen и указателей.

do-while (необязательный фрагмент §6.2.2, файл 05$_{textdowhile}$.c): читает символы, кроме перевода строки, и **сохраняет также сторожевой символ %** — ввод % сохраняет один символ до того, как условие станет ложным; условие проверяет и ввод, и ёмкость (Counter < MaxLength); текст — не C-строка. **continue** (необязательный фрагмент §6.4, файл 06$_{skipletters}$.c): читает пять символов, отличных от перевода строки, и пропускает обработку букв; для 'A' continue обходит printf и попадает на обновление for i++; для '1' printf обрабатывает символ, затем то же обновление; ввод A1b2! оставляет 1, 2 и !.

[Источник: Week-5-Control-Structures, слайды 12–14]

3.2. Savollar недели 5 — вопросы лекции и лаборатории с модельными ответами

S1. Опрос лаборатории: какова роль main в наших программах на C?

Ответ. main — точка входа: выполнение программы начинается с main. stdio.h объявляет printf; cs50.h объявляет get$_{double}$ (и другие помощники ввода CS50).

S2. Чем блок отличается от простого оператора и что можно делать внутри блока?

Ответ. Простой оператор заканчивается ';'. Фигурные скобки объединяют несколько операторов в составной оператор (блок): он может подставляться вместо простого оператора, компилируется как единое целое, внутри можно объявлять переменные (они локальны блоку), блок может быть пустым {}, и после закрывающей скобки не ставится ';'. Блоки вкладываются друг в друга.

S3. Что считается «истиной» в условии C?

Ответ. Условие — выражение (или серия выражений), например n < 3 или x < y || z < y. Любое ненулевое значение ⇒ условие истинно, 0 — ложно. Выражение должно быть числовым (или указателем); в C89/C90 булева типа нет, в C99 bool доступен через <stdbool.h>. Пример: if (str) — строка не null.

S4. Что делает else if и что происходит, если истинны несколько условий?

Ответ. else if добавляет альтернативные пути управления. Условия проверяются по порядку до первого истинного; внутренний оператор выполняется только для него. Если истинны несколько условий, выполняется только первая ветка. Конструкция эквивалентна вложенным if.

S5. К какому if относится else во вложенных if и как это изменить?

Ответ. Без скобок else связывается с ближайшим внутренним if. Чтобы связать else с внешним if, нужно заключить внутренний if в фигурные скобки: if (x % 4 == 0) { if (x % 2 == 0) y = 2; } else y = 1;.

S6. Когда switch удобнее лестницы if-else и какие у него ограничения?

Ответ. switch тестирует одну целочисленную (или символьную) переменную на совпадение с набором case-меток — разных целых константных выражений; для диапазонов и составных условий нужен if. default — ветка «иначе». switch по книге эффективнее длинных лестниц if-else.

S7. Что произойдёт, если забыть break в case?

Ответ. Выполнение «проваливается» (falls through) в следующий case без повторной проверки значения. В мелодии (Example 6.1) пропуск break после case 2 приводил к печати C, затем D для ввода A#. Проваливание бывает и полезным: case 'Y': case 'y': выполняют один код для обоих символов.

S8. Мелодия: почему для символа 'A' программа печатает ноту C?

Ответ. Программа вычисляет ASCII-код символа по модулю 7: код 'A' равен 65; 65 % 7 = 2; case 2 печатает C. Таблица слайда: A → 65 → 2 → C.

S9. Чем do-while отличается от while?

Ответ. В do-while условие вычисляется после каждой итерации, поэтому тело выполняется хотя бы один раз; в конце обязательно ставится точка с запятой: } while (...);. В while условие вычисляется первым — тело может не выполниться ни разу.

S10. Что печатает пара циклов из §6.2.1 и чем они отличаются?

Ответ. Оба печатают 1 2 3. Вариант while объявляет int i = 1; непосредственно перед циклом; вариант for объявляет счётчик в заголовке — он в области видимости только внутри цикла. for собирает инициализацию, условие и обновление в одной строке.

S11. Что делают break и continue и куда именно переходит выполнение?

Ответ. break; выходит из самого внутреннего цикла или switch (например, досрочный выход из do-while при c != 'y'). continue; пропускает остаток тела самого внутреннего цикла и переходит к его условию; в for-цикле лаборатории continue с 'A' обходит printf и попадает на обновление i++, затем — к условию цикла.

S12. Какие три выражения стоят в заголовке for и что можно опустить?

Ответ. Инициализация; условие; инкремент — разделены точками с запятой. Любое из выражений может быть пустым; пустое условие считается истинным. Выражения могут быть составными через оператор «запятая»: for (i = 1, j = 1; i <= n; j *= i, i++) — запятая имеет низший приоритет и вычисляется слева направо, это не запятая между аргументами функции.

S13. Как вручную посчитать длину строки HI! без strlen?

Ответ. Начать счёт n с нуля; пока name[n] != '\0', увеличивать n; остановиться на конечной метке — терминатор не считается. n = 0 читает H, n = 1 читает I, n = 2 читает !, n = 3 достигает NUL — условие ложно, печатается 3.

S14. Что делает статическая переменная внутри функции и чем отличается register?

Ответ. static внутри функции: переменная по-прежнему локальна, но инициализируется только один раз при запуске программы и не переинициализируется при каждом вызове; static вне функции ограничивает видимость текущим файлом (нельзя extern). register — просьба хранить переменную в регистре: только локальные переменные и аргументы; лишние объявления игнорируются; у register-переменной нельзя взять адрес.

3.3. Testlar недели 5

T1. Что делает break внутри switch?

a) завершает всю программу

b) выходит из switch; выполнение продолжается после закрывающей скобки

c) переходит к default

d) пропусрует условие цикла

Ответ: b) выходит из switch; выполнение продолжается после закрывающей скобки

Пояснение: «break leaves the switch» — после printf в case 'B' идёт break, и программа продолжается после switch; break не завершает программу.

T2. Сколько раз гарантированно выполнится тело do-while?

a) ноль раз

b) ровно один раз

c) хотя бы один раз

d) зависит от компилятора

Ответ: c) хотя бы один раз

Пояснение: do-while проверяет условие после каждой итерации («condition evaluated after each iteration; body executed at least once»).

T3. Что печатает for (int i = 1; i <= 3; i++) printf("%d ", i);?

a) 1 2 3

b) 1 2 3 4

c) 0 1 2 3

d) 3 2 1

Ответ: a) 1 2 3

Пояснение: сравнение while/for из §6.2.1 — оба цикла печатают 1 2 3; следующее приращение даёт 4, и тест становится ложным.

T4. Что произойдёт, если опустить условие в заголовке for?

a) ошибка компиляции

b) тело не выполнится ни разу

c) условие будет считаться истинным

d) цикл выполнится ровно один раз

Ответ: c) условие будет считаться истинным

Пояснение: «Expressions can be empty (condition assumed to be \"true\")» — пустое условие for всегда истинно.

T5. Что делает continue;?

a) выходит из цикла
b) пропускает остаток тела внутреннего цикла и переходит к условию
c) перезапускает цикл с инициализации
d) выходит из функции

Ответ: b) пропускает остаток тела внутреннего цикла и переходит к условию

Пояснение: continue; skips rest of innermost loop body, jumping to loop condition; в примере gcd не-делитель i просто пропускается.

T6. Чему равно 65 % 7?

a) 1
b) 2
c) 3
d) 9

Ответ: b) 2

Пояснение: мелодия Example 6.1: код 'A' равен 65, 65 % 7 = 2 — case 2 печатает ноту C.

T7. К какому if относится else без фигурных скобок во вложенных if?

a) к самому внешнему
b) к ближайшему внутреннему
c) к первому в файле
d) ни к какому — это ошибка компиляции

Ответ: b) к ближайшему внутреннему

Пояснение: слайд «Nesting if statements»: else связывается с внутренним if; чтобы связать с внешним, нужны скобки.

T8. Когда проверяется условие цикла while?

a) после тела
b) до тела
c) в середине тела
d) только при первой итерации

Ответ: b) до тела

Пояснение: «Condition evaluated first, so body may never be executed» — тело while может не выполниться ни разу.

T9. В C99 логический тип bool доступен через заголовок…

a) <stdbool.h>
b) <bool.h>
c) <stdio.h>
d) <logic.h>

Ответ: a) <stdbool.h>

Пояснение: «in C99, bool type available (use stdbool.h)»; в C89/C90 булева типа нет — условие просто числовое выражение.

T10. Какой оператор соединяет несколько выражений в заголовке for?

a) точка с запятой

b) запятая

c) &&

d) двоеточие

Ответ: b) запятая

*Пояснение: «Compound expressions separated by commas… Comma: operator with lowest precedence, evaluated left-to-right; not same as between function arguments» — for (i = 1, j = 1; i <= n; j *= i, i++).*

3.4. Atamalar — глоссарий недели 5

Термин (atama)	Значение
Блок / составной оператор	{ … } — объединяет операторы в единицу; компилируется как одно целое; создаёт локальную область видимости; после } точка с запятой не нужна
Ветвление (selection)	выбор пути выполнения: if / else if / else и switch
Условие (condition)	выражение: ненулевое значение — истина, 0 — ложь; числовое выражение или указатель
switch	оператор выбора по целочисленной (символьной) переменной; содержимое — блок
case-метка	целая константная метка-вход в блок switch; метки различны
default	ветка switch «иначе» — когда не совпал ни один case
Проваливание (fall-through)	продолжение выполнения в следующий case при отсутствии break; используется для объединения case 'Y': case 'y':
while	цикл с предусловием: условие вычисляется до тела; тело может не выполниться
for	«счётный» цикл: инициализация; условие; инкремент в заголовке; пустое условие — истинно
do-while	цикл с постусловием: тело хотя бы один раз; в конце — точка с запятой
break	выход из самого внутреннего цикла или switch
continue	пропуск остатка тела внутреннего цикла с переходом к условию (в for — к обновлению)
Сторожевое значение (sentinel)	символ-ограничитель ввода: '#' в мелодии, '%' в примере do-while
Область видимости и затенение (scope, shadowing)	scope — регион действия переменной (обычно блок объявления; глобальные — вне функций); shadowing — внутреннее объявление с тем же именем скрывает внешнее (локальный nmax = 25 скрывает глобальный 20)

4. Неделя 6 — Функции и структуры (Functions and Structures)

Файлы недели: Week-6-Functions-and-Structures.txt (лаборатория «WEEK 6 · CHAPTER 7: From Scratch blocks to a coffee machine») · Lecture$_6$.txt (TashTech Lecture 6 — October 20, 2026)

Неделя 6 соединяет блоки Scratch с настоящими функциями C: определение, прототип, вызов, параметры и аргументы, возвращаемые значения — и добавляет структуры (typedef struct, поля, точка) на классной модели кофейного автомата. Лекция 6 закрепляет управляющие конструкции (goto и

спагетти-код) и полностью закрывает ввод-вывод: стандартный I/O, форматирование printf/scanf, строки, файлы и аргументы командной строки. Ниже — все правила, вопросы, тесты и термины недели.

4.1. Ключевые положения («все правила» недели 6)

Функция: определение, прототип, вызов

Функция — именованный набор инструкций. Читаем заголовок void print$_{\text{helloworld}}$(void): void перед именем — нет возвращаемого результата; (void) — нет входов; фигурные скобки — тело. Важно различать возвращаемое значение и напечатанный вывод.

```
void print_hello_world(void)
{
    printf("Hello, world!\n");
}
// calls inside main
print_hello_world();
print_hello_world();
```

Полная программа: **прототип объявляет функцию до того, как main её вызовет** — заканчивается ';' и не имеет тела. Find it in the code: Prototype — announce the function (объявить); Call — run it now (выполнить); Definition — describe its work (описать работу). Вывод программы — Hello, world! дважды:

```
#include

void print_hello_world(void);

int main(void)
{
    print_hello_world();
    print_hello_world();
    return 0;
}

void print_hello_world(void)
{
    printf("Hello, world!\n");
}
```

Если поменять сообщение внутри функции, оба вызова используют изменённое определение: повторное использование — общее поведение, а не просто меньшая печать.

[Источник: Week-6-Functions-and-Structures, слайды 5–6]

Параметры, аргументы и возвращаемое значение

Одна функция может использовать другую. **Параметр** — имя входа внутри функции (count); **аргумент** — значение, переданное вызовом (3). count локален для print$_{\text{meows}}$; при каждом вызове значение копируется:

```
void print_meow(void)
{
    printf("meow\n");
}

void print_meows(int count)
{
    for (int i = 0; i < count; i++)
    {
        print_meow();
    }
```

```
}

// calls inside main
print_meows(1);
print_meows(3);
```

Возвращаемое значение уходит обратно вызывающему. Книжный пример IntegerRoot — целочисленный квадратный корень: int — тип результата; number — входной параметр; вызывающий (caller) — функция, которая делает вызов (здесь main). IntegerRoot возвращает число; **выводит на экран вызывающий** через printf:

```
int IntegerRoot(int number);

// calls inside main
int first = IntegerRoot(27);
int second = IntegerRoot(16);
printf("%d %d\n", first, second);
```

Для 27: $5 \times 5 = 25 \leq 27$; $6 \times 6 = 36 > 27$ — ответ 5. Для 16 — 4. Программа печатает **5 4**.

[Источник: Week-6-Functions-and-Structures, слайды 7–8]

Структуры: typedef struct и доступ к полям

Определение типа-структуры ставится выше main: struct { … } группирует связанные поля в один тип; int $water_{ml}$; объявляет одно целое поле; typedef даёт типу удобное имя; Stock — имя типа после закрывающей скобки; всё объявление заканчивается ';'. **Поле (field) — одно именованное значение в записи.**

```
typedef struct
{
    int water_ml;
    int milk_ml;
    int beans_g;
    int paper_cups;
} Stock;
```

Создание переменной и работа с полями: значения идут в порядке полей — вода, молоко, зёрна, стаканчики; доступ к одному полю — через точку. Stock (тип, с заглавной) и stock (переменная, строчная) — разные имена:

```
Stock stock = {500, 200, 40, 3};
printf("%d\n", stock.water_ml);
stock.water_ml = 450;
printf("%d\n", stock.water_ml);
```

Поле	Значение после инициализации
$water_{ml}$	500
$milk_{ml}$	200
$beans_{g}$	40
$paper_{cups}$	3

stock после инициализации; вывод программы: 500, затем 450 — меняется только поле $water_{ml}$ [Источник: Week-6-Functions-and-Structures, слайд 10]

[Источник: Week-6-Functions-and-Structures, слайды 9–10]

Копия записи: функция возвращает отдельный Stock

Помощник получает **копию** параметра Stock: изменяет копию и возвращает её; вызывающий решает, куда сохранить результат:

```
Stock stock_after_water_use(
    Stock stock, int amount_ml)
{
    if (amount_ml >= 0 &&
        amount_ml <= stock.water_ml)
    {
        stock.water_ml -= amount_ml;
    }
    return stock;
}

// Caller · stock starts with 500 ml
Stock preview =
    stock_after_water_use(stock, 100);
printf("%d\n", stock.water_ml);
printf("%d\n", preview.water_ml);
stock =
    stock_after_water_use(stock, 50);
printf("%d\n", stock.water_ml);
```

Момент	$stock.water_{ml}$	$preview.water_{ml}$
После первого вызова	500	400
После сохранения в stock	450	400

Первый вызов возвращает изменённую копию, оригинал остаётся 500; следующий вызов на 50 мл стартует от неизменённых 500 и сохраняет 450 в stock [Источник: Week-6-Functions-and-Structures, слайд 11]

[Источник: Week-6-Functions-and-Structures, слайд 11]

Модель кофемашины: $can_{makecoffee}$, $stock_{aftercoffee}$ и Cup

Классная модель: машина имеет water 500 / milk 200 / beans 40 / cups 3; один латте требует 100 / 50 / 10 / 1. **Рецепт — тоже тип-структура (Recipe); переменная latte хранит количества для одного напитка; один принятый заказ использует один стаканчик.** Вычитаем каждый ресурс: 400 мл воды, 150 мл молока, 30 г зёрен, 2 стаканчика.

Функция-решение: bool — результат true/false; >= — «не меньше этого количества»; && — каждое условие должно выполняться; return — передаёт результат вызывающему. «Ровно столько» принимается; одного недостающего стаканчика достаточно, чтобы остановить заказ:

```
bool can_make_coffee(
    Stock stock, Recipe recipe)
{
    return
        stock.water_ml >= recipe.water_ml &&
        stock.milk_ml >= recipe.milk_ml &&
        stock.beans_g >= recipe.beans_g &&
        stock.paper_cups >= recipe.paper_cups;
}
```

Stock для этого латте	Можно приготовить?
500 / 200 / 40 / 3	true
100 / 50 / 10 / 1	true
500 / 200 / 40 / 0	false

Числа — вода/молоко/зёрна/стаканчики [Источник: Week-6-Functions-and-Structures, слайд 13]

Расчёт следующего Stock: если кофе возможен — вычесть все четыре ресурса и вернуть копию; -= вычитает и сохраняет результат; return stock; отправляет обновлённую копию назад. Сравните два вызова:

```
Stock stock_after_coffee(
    Stock stock, Recipe recipe)
{
    if (can_make_coffee(stock, recipe))
    {
        stock.water_ml -= recipe.water_ml;
        stock.milk_ml -= recipe.milk_ml;
        stock.beans_g -= recipe.beans_g;
        stock.paper_cups -= recipe.paper_cups;
    }
    return stock;
}

stock_after_coffee(stock, latte);
// stock is still unchanged

stock =
    stock_after_coffee(stock, latte);
// stock now keeps the returned record
```

Stock вызывающего	Вода / молоко / зёрна / стаканчики
До заказа	500 / 200 / 40 / 3
После сохранения результата	400 / 150 / 30 / 2

Первый вызов отбрасывает возвращённую копию, второй — сохраняет [Источник: Week-6-Functions-and-Structures, слайд 14]

Блок заказа: $\text{print}_{\text{stock}}$ и $\text{print}_{\text{cup}}$ показывают данные; $\text{create}_{\text{cup}}$ строит запись напитка (Cup); false означает «без пены». Если запасов нет ($\text{stock.paper}_{\text{cups}} = 0$), ветка if не выполняет ни одного присваивания — все текущие поля stock и прежний Cup остаются без изменений:

```
print_stock(stock);
if (can_make_coffee(stock, latte))
{
    stock =
        stock_after_coffee(stock, latte);
    cup = create_cup(latte, false);
}
print_stock(stock);
print_cup(cup);
```

[Источник: Week-6-Functions-and-Structures, слайды 12–15]

goto, спагетти-код и обработка ошибок

goto позволяет безусловный переход в произвольное место кода **в пределах той же функции**; место помечается **меткой (label)** — именованной точкой, имеющей ту же форму, что имя переменной, после которого стоит ':'.

```
start:
{
    if (cond)
        goto outside;
    /* some code */
    goto start;
}
outside:
```

```
/* outside block */
```

Дейкстра, «Go To Statement Considered Harmful» (Communications of the ACM 11(3), 1968): избыточный goto создаёт **спагетти-код** — код становится труднее читать и отлаживать; любой код с goto можно переписать без него. Но в C у goto есть законная работа: в C++ и Java есть механизм исключений для восстановления после ошибок, а в C **goto — удобный способ выйти из вложенных блоков** (переход на метку error: пропускает оба блока).

[Источник: Lecture$_6$, слайды 5–7]

Ввод-вывод: putchar/getchar, printf и его форматы, scanf

Средства ввода-вывода даёт стандартная библиотека **<stdio.h>**, а не сам язык. **Текстовый поток** — серия строк, заканчивающихся '\n' (библиотека сама преобразует '\r\n'–'\n'); **двоичный поток** — серия сырых байтов; потоки библиотеки буферизуются.

int putchar(int) помещает символ c в стандартный вывод; возвращает напечатанный символ или EOF при ошибке. **int getchar()** возвращает следующий символ стандартного ввода или EOF при ошибке. Классический пример — перевод в нижний регистр (c = c − 'A' + 'a'; — возможно благодаря целочисленному преобразованию char):

```
int main()
{
    char c;
    while ((c = getchar()) != EOF)
    {
        if (c >= 'A' && c <= 'Z')
            c = c - 'A' + 'a';
        putchar(c);
    }
    return 0;
}
```

Чтобы читать из файла вместо стандартного ввода, используется оператор '<' (*nix): ./a.out < file.txt — это возможность ОС, а не языка.

int printf(char format[], arg1, arg2, ...) — форматированный вывод: принимает переменное число аргументов, возвращает число напечатанных символов. Формат содержит литеральный текст и спецификаторы, начинающиеся с %: **%[flags][width][.precision][length]<type>**.

Тип	Смысл	Пример
d, i	целое	printf("%d",10); / *prints 10* /
u	беззнаковое целое	printf("%u",10); / *prints 10* /
x, X	целое (hex)	printf("%x",10); / *prints 0xa* /
c	символ	printf("%c",'A'); / *prints A* /
s	строка	printf("%s","hello"); / *prints hello* /
f	вещественное	printf("%f",2.3); / *prints 2.3* /
e, E	вещественное (эксп.)	1e3, 1.2E3, 1E-3
%	литеральный %	printf("%d %%",10); / *prints 10%* /

Спецификаторы типа printf [Источник: Lecture$_6$, слайд 12; Lecture$_{71}$, слайд 3]

Модификатор	Смысл
h	интерпретировать как short; использовать с i, d, o, u, x

Модификатор	Смысл
l	интерпретировать как long; использовать с i, d, o, u, x
L	интерпретировать как double; использовать с e, f, g

Модификаторы длины [Источник: Lecture$_6$, слайд 16]

Вызов	Вывод
printf("%d",10)	«10»
printf("%4d",10)	«bb10» (b — пробел: width)
printf("%7s","hello")	«bbhello»
printf("%d,%+d,%+d",10,-10)	«10,+10,-10» (флаг +)
printf("%04d",10)	«0010» (флаг 0)
printf("%-7s","hello")	«hellobb» (флаг –: по левому краю)
printf("%.2f,%.0f",1.141,1.141)	«1.14,1» (precision)
printf("%.2e,%.0e",1.141,100.00)	«1.14e+00,1e+02»
printf("%.4s","hello")	«hell»
printf("%.1s","hello")	«h»

width, flags и precision на примерах [Источник: Lecture$_6$, слайды 13–15]

int scanf(char *format, ...) — входной аналог printf: читает символы стандартного ввода, интерпретируя их по спецификации формата (той же, что у printf); принимает переменное число аргументов; при чтении нескольких элементов каждый считается разделённым пробельными символами; **scanf игнорирует пробельные символы**; возвращает число прочитанных элементов или EOF. Важно: **аргументы должны быть адресами переменных (указателями)**. Примеры соответствия:

printf	scanf
printf("%d",x)	scanf("%d",&x)
printf("%f",f)	scanf("%f",&f)
printf("%s",str)	scanf("%s",str) — & не нужен
printf("%s",str)	scanf("%20s",str) — & не нужен
printf("%s %s",fname,lname)	scanf("%20s %20s",fname,lname)

printf ↔ scanf [Источник: Lecture$_6$, слайд 20]

Форматированные данные можно писать и читать в/из массивов символов: **int sprintf(char string[], char format[], arg1, arg2)** — как printf, но вывод пишется в string (размер не проверяется); **int sscanf(char str[], char format[], arg1, arg2)** — как scanf, но ввод читается из str; обе возвращают число записанных/прочитанных элементов или отрицательное значение при ошибке.

[Источник: Lecture$_6$, слайды 8–21]

Строки как массивы, strcmp, файлы и командная строка

Строки представлены массивами символов; C не ограничивает длину строки — конец строки задаётся нулевым байтом: "hello" — это массив {'h','e','l','l','\0'}. Объявления:

```
char str[] = "hello";   /* compiler takes care of size */
char str[10] = "hello"; /* make sure the array is large enough */
```

```
char str[] = {'h','e','l','l',0};
```

Примечание: чтобы строка содержала символ ", используйте \". Для сравнения строк заголовок **<string.h>** даёт **int strcmp(char s[], char t[])** — сравнение в словарном порядке (строчные буквы идут после заглавных): возвращает значение <0, если s раньше t; 0, если строки одинаковы; >0, если s позже t; strcmp чувствителен к регистру. Примеры: strcmp("A","a") → <0; strcmp("IRONMAN","BATMAN") → >0; strcmp("aA","aA") → ==0; strcmp("aA","a") → >0.

Файлы: FILE *fopen(char name[], char mode[]) открывает поток; режимы "r" (чтение), "w" (запись), "a" (добавление), среди прочих; "b" добавляется для двоичных файлов; fopen возвращает указатель на поток или NULL. int fclose(FILE* fp) закрывает поток (освобождает ресурсы ОС); при завершении программы fclose вызывается автоматически для всех открытых файлов. Стандартные ввод и вывод — тоже FILE *(stdin, stdout); stderr — отдельный поток стандартного вывода ошибок. Посимвольно: int getc(FILE* fp) (getchar — это getc(stdin): #define getchar() getc(stdin)); int putc(int c, FILE *fp) (putchar — putc(c, stdout)); построчно: char* fgets(char line[], int maxlen, FILE *fp) читает одну строку до maxlen символов (включая перевод строки), возвращает указатель на массив или NULL в конце потока; int fputs(char line[], FILE* fp) пишет строку, возвращает 0 при успехе или EOF; int fscanf(FILE *fp, char format[], ...) — как scanf/sscanf, но из потока fp.

Аргументы командной строки: main может принимать параметры, заполняемые при запуске программы: **int main(int argc, char *argv[])**; argc — счётчик аргументов; argv[] — массив указателей на каждый аргумент; аргументы включают и имя программы: ./cat a.txt b.txt даёт argc = 3, argv[0] = "cat", argv[1] = "a.txt", argv[2] = "b.txt"; ./cat даёт argc = 1, argv[0] = "cat".

[Источник: Lecture$_6$, слайды 17–27]

4.2. Savollar недели 6 — вопросы лекции и лаборатории с модельными ответами

S1. Что означают void перед именем функции и (void) в скобках?

Ответ. void перед именем — нет возвращаемого результата; (void) — нет входов. В void print$_{\text{helloworld}}$(void) оба void на своих местах; braces заключают тело. Возвращаемое значение нужно отличать от напечатанного вывода.

S2. Чем прототип отличается от вызова и определения?

Ответ. Прототип объявляет функцию до main: void print$_{\text{helloworld}}$(void); — заканчивается ';' и не имеет тела. Вызов — print$_{\text{helloworld}}$(); — выполняет функцию сейчас. Определение — тело с фигурными скобками — описывает её работу. В одной программе: прототип до main, вызовы внутри main, определение после.

S3. Recap-опрос лекции: что печатает for (int i = 1; i <= 3; i++) printf("%d ", i);?

Ответ. 1 2 3 — счётчик проходит 1, 2, 3; при i = 4 условие i <= 3 ложно. Тот же результат давало сравнение while/for из недели 5.

S4. Recap-опрос: чему равен streak после int streak = 2; int practised = 0; if (practised == 1) streak++; else streak = 0;?

Ответ. streak = 0: practised == 1 ложно (practised равно 0), выполняется ветка else, и streak обнуляется. Любое ненулевое значение — истина, 0 — ложь.

S5. Recap-опрос: без первого break чему равен score? int score = 0; char reaction = 'W'; switch (reaction) { case 'W': score += 2; case 'L': score -= 1; break; }

Ответ. score = 1. Символ 'W' совпадает с case 'W': score += 2 даёт 2; break нет — выполнение проваливается в case 'L' без новой проверки: score -= 1 даёт 1; break завершает switch.

S6. Чем параметр отличается от аргумента?

Ответ. Параметр — имя входа внутри функции: count в void $\text{print}_{\text{meows}}$(int count); это имя локально для функции. Аргумент — значение, которое передаёт вызов: 3 в $\text{print}_{\text{meows}}$(3). При каждом вызове значение копируется в параметр.

S7. Что печатает пара вызовов IntegerRoot(27) и IntegerRoot(16)?

Ответ. 5 4. Для 27: $5 \times 5 = 25 \le 27$, а $6 \times 6 = 36 > 27$, значит целочисленный корень 5; для 16 — 4. IntegerRoot возвращает число, а печатает его вызывающий через printf("%d %d\n", first, second);.

S8. Что такое поле (field) и чем Stock отличается от stock?

Ответ. Поле — одно именованное значение в записи (water_{ml}, milk_{ml}, beans_{g}, $\text{paper}_{\text{cups}}$). Stock — имя типа, введённое typedef после закрывающей скобки; stock — переменная этого типа. Регистр важен: Stock и stock — разные имена.

S9. Почему после Stock preview = $\text{stock}_{\text{afterwateruse}}$(stock, 100); вода в stock всё ещё 500, а в preview — 400?

Ответ. Параметр stock внутри функции — копия: помощник вычитает 100 из копии и возвращает изменённую копию; оригинал в main остаётся 500. Вызывающий выбирает, куда сохранить результат: preview получает 400. После stock = $\text{stock}_{\text{afterwateruse}}$(stock, 50); — новый вызов стартует от неизменённых 500 и сохраняет 450 в stock.

S10. Что меняет присваивание stock = $\text{stock}_{\text{aftercoffee}}$(stock, latte); и что будет без него?

Ответ. С присваиванием stock сохраняет возвращённую запись: 400 / 150 / 30 / 2. Без присваивания (просто $\text{stock}_{\text{aftercoffee}}$(stock, latte);) возвращённая копия отбрасывается — stock не меняется: «stock is still unchanged».

S11. Почему при stock.$\text{paper}_{\text{cups}}$ = 0 заказ не выполняется и ничего не меняется?

Ответ. $\text{can}_{\text{makecoffee}}$ требует, чтобы все четыре ресурса были не меньше требуемых (&& — каждое условие должно выполняться): одного недостающего стаканчика достаточно, чтобы вернуть false. Ветка if не выполняет ни одного присваивания — все поля stock и прежний cup (латте) остаются без изменений.

S12. Чем опасен goto и когда он в C оправдан?

Ответ. Избыточный goto создаёт спагетти-код (Дейкстра, «Go To Statement Considered Harmful», CACM 11(3), 1968): код труднее читать и отлаживать; любой код с goto переписываем без него. В C goto оправдан как удобный способ выхода из вложенных блоков при обработке ошибок — в C++/Java для этого есть исключения, в C их нет.

S13. Почему scanf требует & и когда он не нужен?

Ответ. Аргументы scanf должны быть адресами переменных (указателями): scanf("%d", &x) записывает прочитанное число по адресу x. Для массивов символов (строк) имя массива уже является адресом: scanf("%s", str) — без &; так же scanf("%20s", str) и scanf("%20s %20s", fname, lname). scanf игнорирует пробельные символы и возвращает число прочитанных элементов или EOF.

S14. Что хранят argc и argv?

Ответ. int main(int argc, char *argv[]): argc — количество аргументов; argv[] — массив указателей на каждый аргумент; аргументы включают имя программы. Пример: ./cat a.txt b.txt даёт argc = 3, argv[0] = "cat", argv[1] = "a.txt", argv[2] = "b.txt"; одиночный ./cat даёт argc = 1.

4.3. Testlar недели 6

T1. Что такое прототип функции?

a) тело функции с фигурными скобками

b) объявление функции, заканчивающееся ';' и без тела

c) вызов функции из main

d) переменная внутри функции

Ответ: b) объявление функции, заканчивающееся ';' и без тела

Пояснение: «A prototype declares a function before main calls it. It ends with ; and has no body» — прототип объявляет, вызов выполняет, определение описывает работу.

T2. Сколько входов и возвращаемых значений у void print$_{meow}$(void)?

a) 1 вход, 1 значение

b) нет входов, нет возвращаемого значения

c) нет входов, 1 значение

d) 1 вход, нет значения

Ответ: b) нет входов, нет возвращаемого значения

Пояснение: void перед именем — нет возвращаемого результата; (void) — нет входов.

T3. Что возвращает IntegerRoot(27)?

a) 4

b) 5

c) 6

d) 27

Ответ: b) 5

Пояснение: $5 \times 5 = 25 \le 27$, а $6 \times 6 = 36 > 27$ — целочисленный квадратный корень из 27 равен 5; пара вызовов печатает 5 4.

T4. Как обратиться к полю water$_{ml}$ переменной stock?

a) stock.water$_{ml}$

b) Stock.water$_{ml}$

c) stock->water$_{ml}$

d) stock:water$_{ml}$

Ответ: a) stock.water$_{ml}$

Пояснение: доступ к одному полю — через точку: stock.water$_{ml}$ = 450; (Stock — имя типа, а не переменной).

T5. В typedef struct { … } Stock; имя Stock — это…

a) переменная

b) имя типа, введённое typedef

c) поле структуры

d) функция

Ответ: b) имя типа, введённое typedef

Пояснение: typedef даёт типу удобное имя; Stock стоит после закрывающей скобки — это имя типа, а не переменная и не struct-тег.

T6. После Stock preview = stock$_{afterwateruse}$(stock, 100); чему равно stock.water$_{ml}$?

a) 400
b) 500
c) 450
d) 0

Ответ: b) 500

Пояснение: параметр stock в функции — копия; первый вызов возвращает изменённую копию (preview.water$_{ml}$ = 400), оригинал остаётся 500.

T7. Что напечатает printf("%4d", 10);?

a) «10»
b) «bb10» (b — пробел)
c) «0010»
d) «10bb»

Ответ: b) «bb10» (b — пробел)

Пояснение: width 4 дополняет число пробелами слева; «0010» даёт флаг 0 (%04d), «hellobb» — флаг минус (%-7s).

T8. Что напечатает printf("%.4s", "hello");?

a) hello
b) hell
c) h
d) ello

Ответ: b) hell

Пояснение: точность для строк обрезает вывод до 4 символов; %.1s даст «h», а %.2f — два знака после запятой.

T9. Как правильно прочитать целое в переменную x?

a) scanf("%d", x);
b) scanf("%d", &x);
c) scanf("%d", *x);
d) scanf(%d, &x);

Ответ: b) scanf("%d", &x);

Пояснение: аргументы scanf должны быть адресами переменных (pointers); для массива char имя уже адрес: scanf("%s", str).

T10. Что делает режим "r" функции fopen?

a) открывает файл только для записи
b) открывает файл только для чтения
c) открывает файл для добавления
d) делает файл двоичным

Ответ: b) открывает файл только для чтения

Пояснение: "r" — read only, "w" — write only, "a" — append; "b" добавляется для двоичных файлов; fopen возвращает NULL, если поток не существует.

4.4. Atamalar — глоссарий недели 6

Термин (atama)	Значение
Функция (function)	именованный набор инструкций; void $print_{helloworld}$(void) — определение, $print_{helloworld}$(); — вызов
Прототип (prototype)	объявление функции до main: тип имя(параметры); — заканчивается ';' и не имеет тела
Определение и вызов	определение — тело функции в фигурных скобках («describe its work»); вызов — запуск функции по имени («run it now»): $print_{helloworld}$();
Параметр (parameter)	имя входа внутри функции: count в void $print_{meows}$(int count)
Аргумент (argument)	значение, передаваемое вызовом: 3 в $print_{meows}$(3)
Возвращаемое значение	результат, который функция отправляет вызывающему: int IntegerRoot(int number)
Структура (structure)	тип, группирующий связанные поля в одну запись: struct { … }
Поле (field)	одно именованное значение в записи: $water_{ml}$, $milk_{ml}$, $beans_{g}$, $paper_{cups}$
typedef	даёт структурному типу удобное имя: typedef struct { … } Stock; переменная такого типа — запись (record): Stock stock = {500, 200, 40, 3}; доступ к полям через точку
Метка (label)	именованная точка кода для goto: имя + ':' (start:, outside:, error:)
Спагетти-код	нечитаемый, трудный для отладки код от избытка goto (Дейкстра, 1968)
Текстовый поток (text stream)	серия строк, заканчивающихся '\n'; преобразование '\r\n' берёт на себя библиотека; потоки буферизуются
Спецификатор формата	%[flags][width][.precision][length]<type>: %d, %4d, %04d, %.4s, %e, %%
Аргументы командной строки	int main(int argc, char *argv[]): argc — счётчик, argv — массив указателей на аргументы, включая имя программы

5. Неделя 7 — Указатели, строки и рекурсия (Pointers, Strings, Recursion)

Файлы недели: $Lecture_{7}$1.txt (TashTech Lecture 7 — October 27, 2026: Pointers and Memory Addresses · Arrays and Pointer Arithmetic · Strings · Searching and Sorting) · Recursion-Lecture.txt (WEEK 7 · CHAPTER 8: Recursion)

Неделя 7 — двойная тема. Лекция 7 даёт указатели: адреса переменных, физическая и виртуальная память, разыменование, функции с несколькими выходами, массивы и адресная арифметика, строки и функции string.h, а также базовые алгоритмы поиска и сортировки (linear search, insertion sort, quicksort, binary search). Лаборатория по главе 8 учит рекурсии: базовый случай, меньшая задача, прогресс; трассировка факториала по стеку вызовов, фибоначчи и Ханойская башня. Ниже — все правила, вопросы, тесты и термины недели.

5.1. Ключевые положения («все правила» недели 7)

Указатель: адрес (&) и разыменование (*)

Указатель — адрес переменной в памяти; адрес можно использовать для доступа к переменной и её изменения откуда угодно — это чрезвычайно полезно для структур данных (и знаменито запутыванием кода). **Каждая переменная в памяти имеет адрес.** Не имеют адреса: register-переменные; константы/литералы/defines препроцессора; выражения (если результат не переменная). Адрес берёт оператор **&**; адрес переменной типа t имеет тип **t ***:

```
int n = 4;
double pi = 3.14159;
int *pn = &n;      /* address of integer n */
double *ppi = π /* address of double pi */
```

Разыменование/косвенность — оператор *: доступ к переменной по адресу; разыменованный указатель ведёт себя как обычная переменная. **Нулевой указатель** — 0 (NULL): указатель, который ни на что не ссылается:

```
/* prints "pi = 3.14159\n" */
printf("pi = %g\n", *ppi);

/* pi now equals 7.14159 */
*ppi = *ppi + *pn;
```

Указатели можно явно приводить к любому типу указателя (неявное приведение к/из void * тоже возможно); у разыменованного указателя — новый тип независимо от реальных данных; это может вызвать segmentation faults и другие трудноуловимые ошибки:

```
ppi = (double *)pn; /* pn originally of type (int *) */
```

[Источник: Lecture$_{71}$, слайды 5, 9–11]

Память: физическая и виртуальная, стек и куча

Физическая память — ресурсы, где данные реально хранятся: cache, RAM, жёсткий диск, съёмные носители; у них разные объёмы и скорости доступа; управление памятью — одна из главных функций ОС (она перемещает данные во время выполнения); во встраиваемых процессорах память может быть крайне ограничена. **Виртуальная память** — абстракция ОС: адресное пространство, доступное коду (на 32-битной ОС < 4 ГБ: обычно 2 ГБ для Windows, 3–4 ГБ для Linux); виртуальная память отображается на разные части физической.

Используемые части виртуальной памяти: стек и куча. Стек (stack) — куда попадают объявленные переменные; куча (heap) — куда попадает динамическая память.

Указатели в функциях: несколько выходов и swap

Функции возвращают не более одного значения. Расширенный алгоритм Евклида возвращает gcd(a, b), x и y (ax + by = gcd) — вместо глобальных переменных можно **передавать указатели для нескольких выходов**:

```
int ext_euclid(int a, int b, int *x, int *y);

int x, y, g;
/* assume a, b declared previously */
g = ext_euclid(a, b, &x;, &y;);
```

Чтобы функция могла **изменять переменные вызывающего**, ей передают указатели на них. Классический swap:

```
void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

int a = 5, b = 7;
swap(&a;, &b;);
/* now, a = 7, b = 5 */
```

Обратная ловушка — **переменная, выходящая из области видимости**: возврат указателя на локальный массив недопустим, указатель становится недействительным после исчезновения стекового

кадра функции (компиляторы обычно предупреждают):

```
#include

char * get_message() {
    char msg[] = "Aren't pointers fun?";
    return msg;
}

int main(void) {
    char *string = get_message();
    puts(string);
    return 0;
}
```

[Источник: Lecture$_{71}$, слайды 12–14]

Массивы, sizeof и адресная арифметика

Примитивные массивы реализованы в C как указатель на блок непрерывной памяти. arr ведёт себя как указатель на элемент 0 — но, в отличие от указателя, его нельзя переприсвоить:

```
int arr[8];
int a = arr[0];
int *pa = arr;  /* то же, что int *pa = &arr;[0]; */
```

sizeof(): для примитивных типов/переменных — размер типа в байтах; для примитивных массивов — размер всего массива в байтах. Длина массива — макрос:

```
int s = sizeof(char);  /* == 1 */
double f;              /* sizeof(f) == 8  (64-bit OS) */
int arr[8];            /* sizeof(arr) == 32 (64-bit OS) */
long arr[5];           /* sizeof(arr) == 40 (64-bit OS) */

/* needs to be on one line when implemented */
#define array_length(arr) (sizeof(arr) == 0 ? \
    0 : sizeof(arr) / sizeof((arr)[0]))
```

Адресная арифметика: указатель — не int, но к нему можно прибавить/вычесть целое: **pa + i указывает на arr[i]**; адресное значение увеличивается на i, умноженных на размер типа данных. Если arr[0] имеет адрес 100 (int), то arr[3] — адрес 112 (3 × 4 байта). Вопрос слайда: если char *pc = (char *)pa;, то (int *)(pc + i) == pa + 3 при **i = 12** (3 int = 12 байт по одному char).

[Источник: Lecture$_{71}$, слайды 15–17]

Строки и библиотека string.h

Строки хранятся как **массивы символов, завершающиеся нулевым байтом** (последний символ == '\0'). Со строкой работают как с массивом:

```
char str[] = "This is a string.";
char *pc = str;

*(pc+10) = 'S';
puts(str); /* prints "This is a String." */
```

Функции стандартного заголовка **<string.h>**: копирование — strcpy(), strncpy(); сравнение — strcmp(), strncmp(); длина — strlen(); конкатенация — strcat(), strncat(); поиск символа — strchr(), strrchr(). Точные сигнатуры слайда:

- char *strcpy(strto, strfrom); — копирует strfrom в strto
- char *strncpy(strto, strfrom, n); — копирует n символов из strfrom в strto

- int strcmp(str1, str2); — сравнивает str1 и str2: 0, если равны; положительное, если str1 > str2; отрицательное, если str1 < str2
- int strncmp(str1, str2, n); — сравнивает первые n символов str1 и str2
- int strlen(str); — возвращает длину str
- char *strcat(strto, strfrom); — добавляет strfrom в конец strto
- char *strncat(strto, strfrom, n); — добавляет n символов из strfrom в конец strto
- char *strchr(str, c); — ищет символ c в str: указатель на первое вхождение или NULL
- char *strrchr(str, c); — указатель на последнее вхождение c или NULL

[Источник: Lecture$_{71}$, слайды 18–20]

Поиск и сортировка: linear, insertion, quick, binary

Базовые алгоритмы отлично используют указатели; это лишь несколько примеров, а не курс алгоритмов; сложность описывается **Big-O нотацией**. Линейный поиск возвращает указатель на найденный элемент или NULL:

```
int arr[100]; /* array to search */

int * linear_search(int val) {
    int *parr, *parrend = arr + array_length(arr);
    for (parr = arr; parr < parrend; parr++) {
        if (*parr == val)
            return parr;
    }
    return NULL;
}
```

Простая сортировка вставками — **O(n²)**: идти по массиву, пока не найдён элемент не на месте; вставить его в правильную позицию; повторять до конца массива. Разбито на две функции:

```
/* move previous elements down until insertion point reached */
void shift_element(unsigned int i) {
    int ivalue;
    /* guard against going outside array */
    for (ivalue = arr[i]; i && arr[i-1] > ivalue; i--)
        arr[i] = arr[i-1]; /* move element down */
    arr[i] = ivalue;       /* insert element */
}

/* iterate until out-of-order element found;
   shift the element, and continue iterating */
void insertion_sort(void) {
    unsigned int i, len = array_length(arr);
    for (i = 1; i < len; i++)
        if (arr[i] < arr[i-1])
            shift_element(i);
}
```

Быстрые сортировки (shellsort, mergesort, quicksort…): quicksort — O(n log n) в среднем, O(n^2) в худшем случае; выбрать опорный элемент (pivot); переместить меньшие элементы в одну сторону, большие — в другую; отсортировать стороны по отдельности (**рекурсивный алгоритм**); реализована в стандартной библиотеке как qsort() в stdlib.h. Реализация из K&R:

```
void quick_sort(unsigned int left, unsigned int right) {
    unsigned int i, mid;
    int pivot;
    if (left >= right)
        return; /* nothing to sort */
```

```
    /* pivot is midpoint; move to left side */
    swap(arr + left, arr + (left + right) / 2);
    pivot = arr[mid = left];
    /* separate into side < pivot (left+1 to mid)
       and side >= pivot (mid+1 to right) */
    for (i = left + 1; i <= right; i++)
        if (arr[i] < pivot)
            swap(arr + ++mid, arr + i);
    /* restore pivot position */
    swap(arr + left, arr + mid);
    /* sort two sides */
    if (mid > left)
        quick_sort(left, mid - 1);
    if (mid < right)
        quick_sort(mid + 1, right);
}

quick_sort(0, array_length(arr) - 1);
```

Обсуждение quicksort: в текущем виде неустойчива (равные элементы могут поменяться местами); сортирует на месте — особенно ценно при малой памяти; выбор pivot влияет на производительность (можно случайный); divide and conquer, легко распараллеливается; рекурсивна — в худшем случае может переполнить стек на большом массиве.

Бинарный поиск — O(log n) в среднем и худшем случае: список должен быть отсортирован; сравниваем со средним элементом текущего куска — новый кусок гарантированно вдвое меньше (divide and conquer!). Требует **случайного доступа** к памяти массива: на последовательных данных (жёсткий диск) поиск вперёд-назад расточителен — иногда лучше линейный. Реализована в stdlib.h как bsearch():

```
int * binary_search(int val) {
    unsigned int L = 0, R = array_length(arr), M;
    while (L < R) {
        M = (L + R - 1) / 2;
        if (val == arr[M])
            return arr + M;  /* found */
        else if (val < arr[M])
            R = M;           /* in first half */
        else
            L = M + 1;       /* in second half */
    }
    return NULL; /* not found */
}
```

[Источник: Lecture$_{71}$, слайды 21–32]

Рекурсия: базовый случай, меньшая задача, прогресс

Рекурсия — функция вызывает саму себя для меньшей задачи. Классная аналогия «очередь»: сколько людей от тебя до конца очереди, включая тебя? Спроси следующего и жди; последний отвечает «только я — 1»; ответы возвращаются, каждый добавляет себя: 1 + 1 = 2, 2 + 1 = 3, 3 + 1 = 4. Тот же метод, меньшая задача, пока один случай не решается напрямую.

Вызов функции **приостанавливает вызывающего**: main вызывает double$_{value}$(3) и ждёт; у вызванной функции собственный x = 3; она возвращает 6; main продолжает с result = 6. (Возвращать 6 и напечатать 6 — разные вещи.)

```
int double_value(int x)
{
    return 2 * x;
```

```
}
// Inside main:
int result = double_value(3);
```

Первая рекурсивная функция — обратный отсчёт:

```
void count_down(int n)
{
    if (n == 0)
        return;
    printf("%i\n", n);
    count_down(n - 1);
}
```

Базовый случай: при 0 — остановить этот вызов. **Рекурсивный вызов**: та же функция, меньший вход. Трассировка $count_{down}(3)$: печатает 3, 2, 1 (каждое число на своей строке); $count_{down}(0)$ возвращается без печати — 0 достигается, но не печатается.

Три ингредиента рекурсии: (1) базовый случай — ответ без нового вызова (n == 0); (2) меньшая задача — та же функция на более простом входе ($count_{down}$(n - 1)); (3) прогресс — каждый шаг приближает к базовому случаю (3, 2, 1, 0).

До или после рекурсивного вызова? Печать до вызова — значения появляются при спуске; печать после — печать ждёт возврата меньшего вызова:

```
void count_down(int n)
{
    if (n == 0)
        return;
    printf("%i\n", n);
    count_down(n - 1);
}

void count_up(int n)
{
    if (n == 0)
        return;
    count_up(n - 1);
    printf("%i\n", n);
}
```

	$count_{down}(3)$	**$count_{up}(3)$**
Вывод	3 2 1	1 2 3
Печать	до углубления — при спуске	ждёт возврата меньшего вызова

Порядок печати следует порядку выполнения [Источник: Recursion-Lecture, слайды 14–15]

Базовый случай должен быть достижим: наличие базового случая в коде не гарантирует остановку. Рекурсивный вызов с тем же n печатает 3, 3, 3, 3… — вход никогда не достигает 0; починка — $count_{down}$(n - 1). Отрицательный вход для этой простой функции всё равно сломал бы её — важен контракт входа (n >= 0); неограниченная рекурсия может исчерпать стек:

```
void count_down(int n)
{
    if (n == 0)
        return;
    printf("%i\n", n);
    count_down(n);   /* 3, 3, 3, 3, ... — вход не достигает 0 */
}

/* Repair */
```

```
count_down(n - 1);
```

Перенос-проверка $sum_{to}(n)$ — сумма целых от 1 до n ($sum_{to}(0) = 0$): в пропуске должен стоять n - 1; $sum_{to}(3) = 3 + 2 + 1 + 0 = 6$; в кадре n = 3 ждёт работа «прибавить 3 к меньшему ответу»:

```
int sum_to(int n)
{
    if (n == 0)
        return 0;
    return n + sum_to(n - 1);
}
```

[Источник: Recursion-Lecture, слайды 2–8, 14–17]

Факториал, стек вызовов, фибоначчи и Ханой

Факториал: 3! = 3 × 2 × 1 = 6; часть 2 × 1 — это 2!, поэтому 3! = 3 × 2! — «меньший ответ внутри». Базовый случай: **0! = 1** (если бы база возвращала 0, каждое произведение стало бы нулём):

```
int factorial(int n)
{
    if (n == 0)
        return 1;
    int smaller = factorial(n - 1);
    return n * smaller;
}
```

У каждого вызова свой стековый кадр: кадр (stack frame, activation record) хранит то, что нужно незавершённому вызову: собственные параметры/локальные переменные и точку продолжения. Кадр factorial(3): n = 3; ждёт factorial(2); следующее действие — return 3 * smaller. Новый вызов получает собственный n; вызывающий сохраняет свою незавершённую работу.

Трассировка factorial(3): main ждёт результат; factorial(3) вызывает factorial(2) и ждёт умножения на 3; factorial(2) → factorial(1) → factorial(0). **Активен только верхний вызов.** Базовый случай factorial(0) возвращает 1 — но это не финальный результат: reaching the base case не возвращает 6 в main. Возвраты убирают завершённые кадры: factorial(1) получает smaller = 1, вычисляет 1 × 1 = 1; factorial(2): 2 × 1 = 2; factorial(3): 3 × 2 = 6; main продолжает с result = 6. Порядок вызовов: 3, 2, 1, 0; **возвраты: 1, 1, 2, 6**.

	Рекурсивный факториал	Циклический факториал
Повторяющийся шаг	функция вызывает себя	цикл повторяет тело
Что ждёт	по одному незавершённому вызову на уровень	рекурсивных вызовов нет
Состояние в этом примере	каждый вызов хранит свой n	счётчик и накапливаемое произведение
Результат для 3	6	6

Рекурсия и цикл [Источник: Recursion-Lecture, слайд 18]

Не утверждайте, что рекурсия всегда быстрее или лучше: для факториала цикл и рекурсия дают одинаковый результат с похожим числом умножений; но для самоподобных задач (обход вложенных структур) рекурсивная форма часто проще выражается.

Фибоначчи: 0, 1, 1, 2, 3, 5, 8, … — каждое следующее значение равно сумме двух предыдущих. **Два базовых случая** (первые два значения известны напрямую: F(0) = 0, F(1) = 1); два меньших ответа — сначала полностью заканчивается левый вызов, потом начинается правый, затем они складываются:

```
int fibonacci(int n)
{
    if (n == 0)
        return 0;
```

```
    if (n == 1)
        return 1;
    int left = fibonacci(n - 1);
    int right = fibonacci(n - 2);
    return left + right;
}
```

fibonacci(4) возвращает 3. За fibonacci(4) стоит девять вызовов (читаются слева направо в явном порядке кода: 4, 3, 2, 1, 0, 1, 2, 1, 0): fibonacci(3) → 2; fibonacci(2) → 1 (дважды!); fibonacci(1) → 1 (трижды); fibonacci(0) → 0 (дважды). **fibonacci(2) встречается дважды — тот же ответ вычисляется заново**; выполняется только один вызов в каждый момент; эта простая версия повторяет работу.

- fibonacci(4) → возвращает 3
- fibonacci(3) → возвращает 2
- fibonacci(2) → возвращает 1 (и ещё раз 1 — второй узел)
- fibonacci(1) → возвращает 1 (три вызова)
- fibonacci(0) → возвращает 0 (два вызова)

Ханойская башня (Book Exercise 8.5): перенести все три диска с A на C, используя B. Правила: (1) двигать по одному диску за раз; (2) двигать только верхний диск стержня; (3) никогда не класть больший диск на меньший. Три рекурсивных шага: (1) перенести 2 диска с A на B, используя C — 3 одно-дисковых хода; (2) перенести диск 3 с A на C — 1 ход; (3) перенести 2 диска с B на C, используя A — 3 хода. Итого **3 + 1 + 3 = 7 ходов**. Базовый случай: нет дисков — нет ходов (n == 0). Точная последовательность семи ходов: диск 1 A→C; диск 2 A→B; диск 1 C→B; диск 3 A→C; диск 1 B→A; диск 2 B→C; диск 1 A→C.

[Источник: Recursion-Lecture, слайды 7–13, 19–22]

5.2. Savollar недели 7 — вопросы лекции и лаборатории с модельными ответами

S1. Что такое указатель и что такое нулевой указатель?

Ответ. Указатель — адрес переменной в памяти; адрес позволяет обращаться к переменной и изменять её откуда угодно (особенно полезно для структур данных). Нулевой указатель — 0 (NULL): указатель, который ни на что не ссылается.

S2. Что делают операторы & и *?

Ответ. & — оператор взятия адреса: адрес переменной типа t имеет тип t *(int* pn = &n;). — *оператор разыменования/косвенности: доступ к значению по адресу (printf("pi = %g\n",* ppi); или *ppi* = ppi + *pn;). Разыменованный указатель ведёт себя как обычная переменная.

S3. Как вернуть из функции несколько значений?

Ответ. Передать указатели для дополнительных выходов: int ext_{euclid}(int a, int b, int *x, int* y); вызов g = ext_{euclid}(a, b, &x, &y); записывает x и y по переданным адресам. Функции возвращают не более одного значения — указатели расширяют этот предел.

S4. Что не так с функцией, возвращающей адрес локального массива ($get_{message}$)?

Ответ. char msg[] живёт на стековом кадре $get_{message}$(); после возврата функции кадр исчезает, и указатель становится недействительным (dangling) — puts(string) читает память, которой уже нет. Компиляторы обычно предупреждают. Решения: static-массив, буфер, переданный вызывающим, или malloc-память, которую освободит вызывающий.

S5. Почему swap(int a, int b) не меняет x и y в main и как исправить?

Ответ. C передаёт копии значений: swap меняет собственные локальные копии, переменные main не трогаются. Исправление — указатели: void swap(int *x, int* y) { int temp = *x; x = *y; y = temp; } с вызовом swap(&a, &b); после него a = 7, b = 5.

S6. Что даёт pa + i и почему это не простое сложение чисел?

Ответ. pa + i указывает на arr[i]: адрес увеличивается на i, умноженных на размер типа данных. Указатель — не int: арифметика масштабируется размером элемента (для int шаг 4 байта на 64-битной платформе).

S7. Если arr[0] имеет адрес 100 (массив int), какой адрес у arr[3]? А при char *pc = (char *)pa — какое i даёт (int *)(pc + i) == pa + 3?

Ответ. arr[3] имеет адрес 112: 100 + 3 × 4 байта (размер int). Для char-указателя шаг — 1 байт, поэтому i = 12: 3 int = 12 байт.

S8. Какие функции работы со строками есть в string.h?

Ответ. Копирование: strcpy(strto, strfrom), strncpy(strto, strfrom, n). Сравнение: strcmp(str1, str2) — 0 при равенстве, >0 если str1 > str2, <0 если str1 < str2; strncmp — первые n символов. Длина: strlen(str). Конкатенация: strcat, strncat. Поиск символа: strchr — первое вхождение, strrchr — последнее (NULL, если не найден).

S9. Назовите сложности четырёх алгоритмов лекции 7.

Ответ. Сортировка вставками — $O(n^2)$. Quicksort — O(n log n) в среднем, $O(n^2)$ в худшем случае (реализована как qsort() в stdlib.h). Линейный поиск — посещает в среднем половину элементов. Бинарный поиск — O(log n) в среднем и худшем случае (требует отсортированных данных и случайного доступа; реализован как bsearch()).

S10. Какие три ингредиента рекурсии?

Ответ. (1) Базовый случай — ответ без другого вызова (n == 0). (2) Меньшая задача — та же функция на более простом входе ($count_{down}$(n - 1)). (3) Прогресс — каждый допустимый рекурсивный путь приближает к базовому случаю (3, 2, 1, 0).

S11. Что печатает $count_{down}$(3) и $count_{up}$(3) и почему вывод разный?

Ответ. $count_{down}$(3) печатает 3 2 1, $count_{up}$(3) — 1 2 3. В $count_{down}$ печать стоит до рекурсивного вызова — значения появляются при спуске. В $count_{up}$ печать стоит после вызова — она ждёт возврата меньшего вызова и выполняется при возврате. Обе функции посещают n = 0 и останавливаются.

S12. Почему $count_{down}$ с рекурсивным вызовом $count_{down}$(n) зацикливается и как починить?

Ответ. Вход никогда не достигает 0: каждый вызов получает тот же n, печатается 3, 3, 3, 3… — существования базового случая в коде недостаточно, каждый путь должен делать прогресс. Починка: $count_{down}$(n - 1). Неограниченная рекурсия может исчерпать пространство стека.

S13. Что возвращает fibonacci(4) и почему fibonacci(2) встречается дважды?

Ответ. fibonacci(4) возвращает 3. Дерево вызовов содержит девять вызовов: 4, 3, 2, 1, 0, 1, 2, 1, 0 — оба поддерева fibonacci(2) вычисляют один и тот же ответ независимо: вторая ветвь не помнит первую. Выполняется только один вызов в каждый момент; простая версия повторяет работу.

S14. Ханойская башня: назовите три рекурсивных шага и общее число ходов для трёх дисков.

Ответ. (1) Перенести 2 диска с A на B, используя C — 3 хода; (2) перенести диск 3 с A на C — 1 ход; (3) перенести 2 диска с B на C, используя A — 3 хода. Итого 3 + 1 + 3 = 7 ходов. Базовый случай: нет дисков — нет ходов (n == 0). Правила: один диск за раз, только верхний диск, никогда больший на

меньший.

5.3. Testlar недели 7

T1. Что делает оператор & применительно к переменной?

a) возвращает адрес переменной
b) разыменовывает указатель
c) умножает переменную саму на себя
d) объявляет указатель

Ответ: a) возвращает адрес переменной

*Пояснение: «How to find an address of a variable? The & operator»: int *pn = &n; — адрес целого n.*

T2. Какой тип имеет выражение &n, если int n;?

a) int
b) int *
c) double *
d) void

Ответ: b) int *

Пояснение: «Address of a variable of type t has type t » — адрес int имеет тип int .

T3. Что делает *ppi* = ppi + *pn;?

a) изменяет значение переменной pi через указатель
b) изменяет адрес ppi
c) приводит к ошибке компиляции
d) ничего не делает

Ответ: a) изменяет значение переменной pi через указатель

Пояснение: слева — запись по адресу (pi теперь 7.14159), справа — чтение значений через разыменование; разыменованный указатель ведёт себя как обычная переменная.

T4. Что из перечисленного НЕ имеет адреса?

a) переменная в памяти
b) register-переменная
c) элемент массива
d) указатель

Ответ: b) register-переменная

Пояснение: «What doesn't have an address? register variables; constants/literals/preprocessor defines; expressions (unless result is a variable)».

T5. Чему равен sizeof(arr) для int arr[8]; на 64-битной платформе?

a) 8
b) 32
c) 4
d) 64

Ответ: b) 32

Пояснение: для примитивных массивов sizeof даёт размер массива в байтах: 8 элементов × 4 байта = 32; long arr[5] даёт 40.

T6. Куда попадают объявленные переменные, а куда — динамическая память?

a) обе в стек

b) обе в кучу

c) объявленные — в стек, динамическая — в кучу

d) объявленные — в кучу, динамическая — в стек

Ответ: c) объявленные — в стек, динамическая — в кучу

Пояснение: «Usable parts of virtual memory: stack and heap. stack: where declared variables go; heap: where dynamic memory goes».

T7. strcmp(str1, str2) возвращает 0, когда…

a) str1 больше str2

b) строки равны

c) str1 меньше str2

d) строки разной длины

Ответ: b) строки равны

Пояснение: strcmp возвращает 0, если равны; положительное, если str1 > str2; отрицательное, если str1 < str2.

T8. Что делает strchr(str, c)?

a) ищет подстроку в строке

b) возвращает указатель на первое вхождение символа c или NULL

c) возвращает последнее вхождение c

d) считает длину строки

Ответ: b) возвращает указатель на первое вхождение символа c или NULL

Пояснение: strchr — первое вхождение, strrchr — последнее; обе возвращают NULL, если символ не найден.

T9. Базовый случай рекурсии — это…

a) ответ, который получается без нового рекурсивного вызова

b) первый вызов функции

c) цикл внутри функции

d) последний возврат из main

Ответ: a) ответ, который получается без нового рекурсивного вызова

Пояснение: «Base case: an answer without another call» — в $count_{down}$ это n == 0: return;. Другие ингредиенты: меньшая задача и прогресс.

T10. Что печатает $count_{down}(3)$?

a) 3 2 1

b) 1 2 3

c) 3 3 3

d) 0 1 2 3

Ответ: a) 3 2 1

Пояснение: печать стоит до рекурсивного вызова — значения появляются при спуске; $count_{up}(3)$ с печатью после вызова даёт 1 2 3.

5.4. Atamalar — глоссарий недели 7

Термин (atama)	Значение
Указатель (pointer)	адрес переменной в памяти; доступ/изменение переменной откуда угодно; основа структур данных
Оператор & (address-of)	возвращает адрес переменной; адрес переменной типа t имеет тип t *
Разыменование (*) (dereferencing/indirection)	чтение/запись значения по адресу: *ppi* = ppi + *pn;
Нулевой указатель (NULL)	указатель 0, который ни на что не ссылается
Виртуальная память	абстракция ОС: адресное пространство, доступное коду; отображается на физическую память
Стек (stack)	часть виртуальной памяти, куда попадают объявленные переменные (и кадры вызовов)
Куча (heap)	куда попадает динамическая память
Висячий указатель (dangling)	указатель на переменную, вышедшую из области видимости (возврат адреса локального массива)
Адресная арифметика	pa + i указывает на arr[i]; адрес растёт на i × размер типа; arr[0] по адресу 100 → arr[3] по 112
Нуль-терминатор ('\0') и strlen	нулевой байт, завершающий C-строку; strlen(str) возвращает длину без терминатора — для копирования нужен запас +1 байт
Базовый случай (base case)	ответ без нового рекурсивного вызова: n == 0 → return 1 в факториале; два базовых случая у фибоначчи
Стек вызовов / кадр (stack frame, activation record)	запись о незавершённом вызове: собственные параметры, локальные переменные, точка продолжения
O-нотация (Big-O)	порядок роста алгоритма: вставки $O(n^2)$; quicksort O(n log n) в среднем, $O(n^2)$ в худшем; бинарный поиск O(log n)
Ханойская башня (Tower of Hanoi)	классическая рекурсивная задача: 3 диска с A на C через B за 3 + 1 + 3 = 7 ходов; базовый случай — нет дисков, нет ходов

6. Bug Hunts — все ошибки и их исправления

Три рабочих листа курса — «охота за ошибками»: **bug_{huntc}.txt** (условные конструкции и функции), **$bug_{huntloops}$.txt** (циклы) и **$bug_{huntpointersstrings}$.txt** (указатели и строки, с ключом инструктора $bug_{huntpointersstringsanswerkey}$.txt). Правила игры из листов: работать в парах; в каждой программе найти ошибку, объяснить, почему она неправильна, и исправить; скомпилировать и протестировать с предложенными входами. Совет ключа: компилируйте с gcc -g -Wall -fsanitize=address и прогоняйте исправления — сообщения санитайзера отличная тема для обсуждения. Ниже каждая задача разобрана полностью: дословный листинг, объяснение и исправленный код.

6.1. Bug Hunt C

Лист «BUG HUNT: CONDITIONALS AND FUNCTIONS (C)» — шесть задач. Быстрый раунд (10 минут): задачи 1, 2, 5; задачи для обсуждения: 3 и 4 (ошибки видны только на определённых входах); C-специфичные уроки: 2, 5 и 6 (ошибки компилятора и неинициализированные переменные). Ключ инструктора включён в файл и приведён ниже.

B1. Grade checker — порядок условий

Ожидание: 95 → 'A', 85 → 'B', 75 → 'C', 65 → 'D', 40 → 'F'. Листинг с ошибками (дословно):

```
#include

char grade(int score) {
     if (score >= 90) {
        return 'A';
     else if (score >= 80) {
        return 'D';
    } else if (score >= 70) {
        return 'C';
    } else if (score >= 60) {
        return 'B';
    } else
    } else {
        return 'F';
    }
}

int main(void) {
char student1 = grade(95);
char student2 = grade(78
    printf("%c\n", grade(95));
    puts("hello");
    return 0;
}
```

B1. Что не так в программе grade и как её исправить?

Ответ. По ключу: условия проверяются в неправильном порядке — сначала должна идти самая высокая граница (>= 90, затем >= 80, затем >= 70, затем >= 60); из-за неверного порядка score = 95 первым делом попадает в ветку (score >= 60) и возвращает 'D', остальные ветки недостижимы. В напечатанном листинге порядок перепутан вместе с буквами (ветка >= 80 возвращает 'D', а >= 60 — 'B') и сломан синтаксис: после return 'A'; нет закрывающей скобки, продублирован else, вызов grade(78 не закрыт — программа даже не скомпилируется. Исправление: проверить границы по убыванию, вернуть каждой ветке свою букву и восстановить скобки:

```
#include

char grade(int score) {
    if (score >= 90) {
        return 'A';
    } else if (score >= 80) {
        return 'B';
    } else if (score >= 70) {
        return 'C';
    } else if (score >= 60) {
        return 'D';
    } else {
        return 'F';
    }
}

int main(void) {
    char student1 = grade(95);
    char student2 = grade(78);
    printf("%c\n", grade(95));
    puts("hello");
    return 0;
}
```

[Источник: bug$_{huntc}$.txt, PROBLEM 1 + ключ]

B2. Even or odd — сравнение и потерянный return

Ожидание: is$_{even}$(4) → 1, is$_{even}$(7) → 0. Листинг с ошибками (дословно):

```
#include

int is_even(int n) {
    int result;
    if (n % 2 == 0) {
        result = 0;
    } else {
        result = 1;
    }
    return result;
}


int main(void) {
result = 5;
    printf("%d\n", is_even(4));
    return 0;
}
```

B2. Что не так в программе is$_{even}$ и как её исправить?

Ответ. Ключ называет две ошибки: (a) «n % 2 = 0» — присваивание вместо сравнения, нужен == (GCC сообщает об ошибке: нельзя присвоить выражению); (b) функция не возвращает result — без return result; поведение не определено. В напечатанном листинге сравнение уже исправлено на ==, но ветки перепутаны местами: чётному числу присваивается 0, а нечётному 1, а main обращается к result вне функции — переменной result в main не существует. Правильно: чётное → 1, нечётное → 0, вернуть result, убрать строку result = 5; из main:

```
#include

int is_even(int n) {
    int result;
    if (n % 2 == 0) {
        result = 1;
    } else {
        result = 0;
    }
    return result;
}

int main(void) {
    printf("%d\n", is_even(4));
    return 0;
}
```

[Источник: bug$_{huntc}$.txt, PROBLEM 2 + ключ]

B3. Leap year — недостижимая ветка % 400

Ожидание: 1900 → 0, 2000 → 1, 2024 → 1, 2023 → 0. Листинг (дословно):

```
#include

int is_leap(int year) {
    if (year % 4 == 0) {
        return 1;
```

```
    } else if (year % 100 == 0) {
        return 0;
    } else if (year % 400 == 0) {
        return 1;
    } else {
        return 0;
    }
}

int main(void) {
    printf("%d\n", is_leap(1900));
    printf("%d\n", is_leap(2000));
    return 0;
}
```

B3. Почему $is_{leap}(1900)$ возвращает 1, хотя 1900 — обычный год?

Ответ. Любой год, кратный 4, немедленно возвращает 1, поэтому 1900 ошибочно признаётся високосным, а ветка % 400 не может выполниться никогда — она стоит после % 4. Ключ предписывает: проверять % 400 первым, затем % 100, затем % 4 — или использовать компактное выражение (то же условие, что в лаборатории недели 4): return (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;.

```
#include

int is_leap(int year) {
    if (year % 400 == 0) {
        return 1;
    } else if (year % 100 == 0) {
        return 0;
    } else if (year % 4 == 0) {
        return 1;
    } else {
        return 0;
    }
}

/* или одной строкой, по ключу и лаборатории недели 4: */
int is_leap(int year) {
    return (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
}
```

[Источник: bug_{huntc}.txt, PROBLEM 3 + ключ]

B4. Biggest of three — равенство наибольших

Ожидание: biggest(5, 5, 3) → 5. Листинг (дословно):

```
#include

int biggest(int a, int b, int c) {
    if (a > b && a > c) {
        return a;
    } else if (b > a && b > c) {
        return b;
    } else {
        return c;
    }
}

int main(void) {
    printf("%d\n", biggest(5, 5, 3));
    printf("hello");
```

```
    puts("hello")
    return 0;
}
```

B4. Почему biggest(5, 5, 3) возвращает 3?

Ответ. При равенстве наибольших значений (5 и 5) ни одно строгое сравнение «>» не истинно: оба первых условия ложны, выполнение проваливается в else и возвращает c = 3. По ключу: использовать >= в сравнениях. Дополнительно в main пропущена точка с запятой после puts("hello") — синтаксическая ошибка, её тоже нужно исправить:

```
#include

int biggest(int a, int b, int c) {
    if (a >= b && a >= c) {
        return a;
    } else if (b >= a && b >= c) {
        return b;
    } else {
        return c;
    }
}

int main(void) {
    printf("%d\n", biggest(5, 5, 3));
    printf("hello");
    puts("hello");
    return 0;
}
```

[Источник: bug_{huntc}.txt, PROBLEM 4 + ключ]

B5. Double and add — void-функция в выражении

Ожидание: total = 16 (10 + 6). Листинг (дословно):

```
#include

void doubled(int x) {
    printf("%d\n", x * 2);
}

int main(void) {
    int total = doubled(5) + doubled(3);
    printf("%d\n", total);
    return 0;
}
```

B5. Почему int total = doubled(5) + doubled(3); не компилируется?

Ответ. doubled объявлена как void — она ничего не возвращает и не может использоваться в выражении; она печатает вместо возврата. Компилятор сообщает: «void value not ignored as it ought to be». По ключу: сменить тип возвращаемого значения на int и использовать return x * 2;, печать перенести в main. Тогда total = 10 + 6 = 16, как и ожидается:

```
#include

int doubled(int x) {
    return x * 2;
}

int main(void) {
```

```
    int total = doubled(5) + doubled(3);
    printf("%d\n", total);
    return 0;
}
```

[Источник: bug$_{huntc}$.txt, PROBLEM 5 + ключ]

B6. Member discount — неинициализированная переменная

Ожидание: apply$_{discount}$(100, 0) → 100.0, apply$_{discount}$(100, 1) → 80.0. Листинг (дословно):

```
#include

double apply_discount(double price, int is_member) {
    double discount;
    if (is_member) {
        discount = 0.2;
    }
    return price * (1 - discount);
}

int main(void) {
    printf("%.2f\n", apply_discount(100, 0));
    return 0;
}
```

B6. Какое значение имеет discount при is$_{member}$ = 0 и чем это опасно?

Ответ. При is$_{member}$ = 0 ветка if не выполняется, и discount остаётся неинициализированной — результатом будет мусор (undefined behavior). Это C-специфичная ловушка: в C нет автоматического значения по умолчанию. По ключу: инициализировать переменную (double discount = 0.0;) или добавить ветку else:

```
#include

double apply_discount(double price, int is_member) {
    double discount = 0.0;
    if (is_member) {
        discount = 0.2;
    }
    return price * (1 - discount);
}

int main(void) {
    printf("%.2f\n", apply_discount(100, 0));
    return 0;
}
```

[Источник: bug$_{huntc}$.txt, PROBLEM 6 + ключ]

6.2. Bug Hunt Loops

Лист озаглавлен «C++ BUG HUNT: LOOPS» и предполагает #include <iostream>, <vector> и using namespace std; фрагменты используют и printf, и cout — листинг сохранён дословно. Правила листа: каждая программа содержит ровно ОДНУ ошибку, связанную с циклами; прочитать ожидаемое поведение; найти и подчеркнуть строку; объяснить одним предложением; написать исправление. Официального ключа в файле нет — разборы ниже составлены по ожидаемому поведению каждого фрагмента.

L1. Сумма оценок — задача-проверка

Ожидание: печатает "Sum: 405". Листинг (дословно):

```
int scores[5] = {90, 85, 70, 100, 60};
int sum = 0;
for (int i = 0; i < 5; i++) {
    sum += scores[i];
}
printf("Sum: %d", sum);
```

L1. Найдите ошибку в этом цикле.

Ответ. Это задача-проверка: как напечатано, цикл корректен — i проходит индексы 0–4, каждый элемент прибавляется один раз, и сумма 90 + 85 + 70 + 100 + 60 = 405, поэтому программа печатает ровно "Sum: 405", как и ожидается. Урок: прежде чем «чинить» работающий код, просчитайте результат вручную — не всякий фрагмент с циклом содержит баг (единственный формальный недочёт — в printf нет перевода строки \n, на результат он не влияет). Исправление не требуется:

```
int scores[5] = {90, 85, 70, 100, 60};
int sum = 0;
for (int i = 0; i < 5; i++) {
    sum += scores[i];
}
printf("Sum: %d\n", sum); /* добавлен \n — по желанию */
```

[Источник: $bug_{huntloops}$.txt, BUG 1]

L2. Точка с запятой после for

Ожидание: печатает сумму 1..10, то есть 55. Листинг (дословно):

```
int sum = 0;
int i;
for (i = 1; i <= 10; i++); {
    sum += i;
}
  printf("%d", sum);
```

L2. Почему печатается 11 вместо 55?

Ответ. Точка с запятой сразу после заголовка for делает телом цикла пустой оператор: цикл десять раз «прокручивается» впустую, доводя i до 11, и лишь затем блок { sum += i; } выполняется один раз с i = 11 — sum = 11. Исправление: убрать ';' после for (i = 1; i <= 10; i++):

```
int sum = 0;
int i;
for (i = 1; i <= 10; i++) {
    sum += i;
}
printf("%d", sum);
```

[Источник: $bug_{huntloops}$.txt, BUG 2]

L3. Потерянная открывающая скобка

Ожидание: печатает "Count: 0" … "Count: 4" и останавливается. Листинг (дословно):

```
for (int count = 0; count < 5; ++count)
    printf("Count: %d", count);
}
```

L3. Что мешает этой программе напечатать Count: 0 … 4?

Ответ. После заголовка for нет открывающей скобки '{', поэтому телом цикла является только printf — само по себе это дало бы верный вывод, но одинокая закрывающая '}' в конце — синтаксическая ошибка: программа не скомпилируется. Исправление: добавить '{' после заголовка (или убрать лишнюю '}'):

```
for (int count = 0; count < 5; ++count) {
    printf("Count: %d", count);
}
```

[Источник: bug$_{huntloops}$.txt, BUG 3]

L4. printf без аргумента

Ожидание: печатает "1 2 3 4 Found 5!" и останавливается. Листинг (дословно):

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        printf("Found 5!");
        break;
    }
    printf("%i ");
}
```

L4. Почему ожидаемые числа 1 2 3 4 не появляются?

Ответ. В printf("%i "); есть спецификатор формата %i, но не передан аргумент — поведение не определено (обычно печатается мусор вместо чисел). Исправление: передать i — printf("%i ", i);. Тогда цикл напечатает 1 2 3 4, при i == 5 — "Found 5!" и break завершит цикл:

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        printf("Found 5!");
        break;
    }
    printf("%i ", i);
}
```

[Источник: bug$_{huntloops}$.txt, BUG 4]

L5. Беззнаковый счётчик не бывает отрицательным

Ожидание: печатает "5 4 3 2 1 0 Liftoff!" и программа завершается. Листинг (дословно):

```
for (unsigned int i = 5; i >= 0; i--) {
    printf("%d ", i);
}
printf("Liftoff!");
```

L5. Почему "Liftoff!" никогда не печатается?

Ответ. i объявлена как unsigned int — она не бывает меньше нуля, поэтому условие i >= 0 всегда истинно: после 0 оператор i-- «заворачивает» счётчик в огромное число (принцип беззнаковой арифметики из недели 4: unsigned wraparound), и цикл становится бесконечным. Исправление: знаковый счётчик int, для которого условие i >= 0 станет ложным после 0:

```
for (int i = 5; i >= 0; i--) {
    printf("%d ", i);
}
printf("Liftoff!");
```

[Источник: bug$_{huntloops}$.txt, BUG 5]

L6. erase во время итерации (C++)

Ожидание: удалить все чётные числа, оставив {1, 3, 5}. Листинг (дословно):

```
vector nums = {1, 2, 3, 4, 5, 6};
for (auto it = nums.begin(); it != nums.end(); ++it) {
    if (*it % 2 == 0) {
        nums.erase(it);
    }
}
for (int n : nums) {
    cout << n << " ";
}
cout << endl;
```

L6. Что происходит с итератором после nums.erase(it)?

Ответ. nums.erase(it) делает итератор it недействительным (инвалидация итератора), а цикл сразу делает ++it по невалидному итератору — неопределённое поведение. Исправление: присваивать итератор, возвращённый erase (он указывает на следующий элемент), и инкрементировать его только когда удаления не было:

```
vector nums = {1, 2, 3, 4, 5, 6};
for (auto it = nums.begin(); it != nums.end(); ) {
    if (*it % 2 == 0) {
        it = nums.erase(it);
    } else {
        ++it;
    }
}
for (int n : nums) {
    cout << n << " ";
}
cout << endl;
```

[Источник: $bug_{huntloops}$.txt, BUG 6]

L7. Сравнение double на равенство

Ожидание: печатает 0, 0.1, 0.2, … до 0.9 и останавливается. Листинг (дословно):

```
for (double x = 0.0; x != 1.0; x += 0.1) {
    printf("%f ", x);
}
```

L7. Почему цикл не останавливается на 0.9?

Ответ. 0.1 не представимо точно в двоичной вещественной арифметике; после десяти прибавлений накопленная погрешность «промахивается» мимо точного 1.0, условие x != 1.0 остаётся истинным, и цикл уходит за 0.9 (то же правило, что в неделе 4: сравнение == на float/double ненадёжно из-за конечной точности). Исправление: условие с отношением порядка x < 1.0:

```
for (double x = 0.0; x < 1.0; x += 0.1) {
    printf("%f ", x);
}
```

[Источник: $bug_{huntloops}$.txt, BUG 7]

L8. Затенение переменной total

Ожидание: печатает "Total: 15". Листинг (дословно):

```
int total = 0;
for (int i = 1; i <= 5; i++) {
    int total = 0;
```

```
        total += i;
    }
    printf("Total: %d", total);
```

L8. Почему печатается "Total: 0", а не 15?

Ответ. Внутри цикла объявлена новая переменная total, которая затеняет внешнюю (shadowing — тот же механизм, что с локальным nmax = 25 из лекции 5): каждая итерация создаёт свежую внутреннюю total, прибавляет к ней i и выбрасывает её. Внешняя total ни разу не изменяется. Исправление: убрать внутреннее объявление int total = 0; — тогда total += i; будет обновлять внешнюю переменную: 1 + 2 + 3 + 4 + 5 = 15:

```
int total = 0;
for (int i = 1; i <= 5; i++) {
    total += i;
}
printf("Total: %d", total);
```

[Источник: $bug_{huntloops}$.txt, BUG 8]

L9. Инкремент не той переменной

Ожидание: таблица умножения 3×3: строки 1 2 3 / 2 4 6 / 3 6 9. Листинг (дословно):

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; i++) {
        cout << i * j << "\t";
    }
    cout << endl;
}
```

L9. Почему внутренний цикл никогда не заканчивается?

Ответ. Внутренний цикл инкрементирует i вместо j: j навсегда остаётся 1, условие j <= 3 всегда истинно, а i растёт бесконечно — внутренний цикл зацикливается (и портит счётчик внешнего). Исправление: j++ во внутреннем заголовке:

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        cout << i * j << "\t";
    }
    cout << endl;
}
```

[Источник: $bug_{huntloops}$.txt, BUG 9]

L10. Поиск 7 в сетке — обрыв листинга [Hard]

Ожидание: найти ПЕРВОЕ вхождение 7 в сетке и напечатать "First 7 found in row 0". Листинг в файле задания обрывается (дословно, весь доступный фрагмент):

```
int grid[3][3] = {
    {1, 7, 3},
```

L10. Листинг оборван — какой урок заложен в эту задачу?

Ответ. Полный код задачи в файле курса отсутствует (текст обрывается сразу после начала объявления сетки), поэтому дословное исправление невозможно. По видимому фрагменту: 7 стоит в строке 0, столбце 1 — ожидаемый вывод "First 7 found in row 0". Правило, которое проверяет эта задача: break выходит только из самого внутреннего цикла (лекция 5: «break exits innermost loop or switch») — при поиске по сетке break во внутреннем цикле по столбцам не остановит внешний цикл по строкам. Чтобы завершить поиск после первой находки, используют флаг «найдено» или goto-выход из вложенных

блоков (лекция 6: «goto provides a convenient way to exit from nested blocks»). Разбирайте эту задачу как обсуждение правил, а не как компилируемый фрагмент.

[Источник: bug$_{huntloops}$.txt, BUG 10 — фрагмент обрывается в исходном файле]

6.3. Bug Hunt Pointers & Strings

Лист «C BUG HUNT: POINTERS AND STRINGS» — двенадцать задач. Все ошибки ключа проверены компиляцией с gcc и прогоном под AddressSanitizer/UBSan; многие из них — undefined behavior, поэтому точные симптомы могут отличаться на разных компиляторах, флагах и машинах. Ключ инструктора (bug$_{huntpointersstringsanswerkey}$.txt) — авторитетный источник исправлений; ниже каждая задача: листинг дословно, объяснение из ключа и исправленный код.

P1. Отсутствует нуль-терминатор

```
char s[5] = {'h', 'e', 'l', 'l', 'o'};
printf("%s\n", s);
```

P1. Ожидается "hello" — что напечатает этот код и почему?

Ответ. В s ровно 5 ячеек на 5 букв — места для '\0' не остаётся. printf("%s") продолжает читать память за массивом, пока не встретит нулевой байт: вывод непредсказуем. Исправление по ключу: char s[6] = {'h','e','l','l','o','\0'}; или char s[] = "hello"; (компилятор сам добавит терминатор и размер):

```
char s[6] = {'h', 'e', 'l', 'l', 'o', '\0'};
printf("%s\n", s);

/* или: */
char s2[] = "hello";
printf("%s\n", s2);
```

[Источник: bug$_{huntpointersstrings}$.txt, BUG 1 + ключ]

P2. sizeof на указателе

```
char *s = "hello";
printf("%zu\n", sizeof(s));
```

P2. Ожидается длина строки 5 — что вернёт sizeof(s)?

Ответ. s — это char *, поэтому sizeof(s) — размер самого указателя (8 на 64-битной машине), а не длина строки: напечатается 8. Длину строки возвращает strlen(s). Совет ключа: sizeof на настоящем массиве (char a[] = "hello") даёт 6, включая '\0'.

```
char *s = "hello";
printf("%zu\n", strlen(s));
```

[Источник: bug$_{huntpointersstrings}$.txt, BUG 2 + ключ]

P3. Сравнение строк через ==

```
char a[] = "cat";
char b[] = "cat";
if (a == b)
    puts("Same");
else
    puts("Different");
```

P3. Почему печатается "Different", хотя текст одинаковый?

Ответ. Оператор == сравнивает адреса двух массивов — это два разных объекта в памяти, поэтому сравнение ложно. Содержимое строк сравнивает strcmp: if (strcmp(a, b) == 0) — тогда напечатается

"Same".

```
char a[] = "cat";
char b[] = "cat";
if (strcmp(a, b) == 0)
    puts("Same");
else
    puts("Different");
```

[Источник: bug$_{huntpointersstrings}$.txt, BUG 3 + ключ]

P4. Буфер слишком мал для strcpy

```
char dest[5];
strcpy(dest, "hello");
printf("%s\n", dest);
```

P4. Сколько байт нужно dest для копирования "hello" и что происходит здесь?

Ответ. "hello" требует 6 байт: 5 букв плюс '\0', а dest вмещает 5 — strcpy пишет один байт за концом массива (переполнение буфера). Исправление: char dest[6]; или больше.

```
char dest[6];
strcpy(dest, "hello");
printf("%s\n", dest);
```

[Источник: bug$_{huntpointersstrings}$.txt, BUG 4 + ключ]

P5. Изменение строкового литерала

```
char *s = "hello";
s[0] = 'J';
printf("%s\n", s);
```

P5. Почему s[0] = 'J'; приводит к падению программы?

Ответ. "hello" — литерал в памяти только для чтения; s лишь указывает на него, и запись s[0] = 'J' пытается изменить read-only память — обычно сегментация (segfault). Исправление по ключу: char s[] = "hello"; — так создаётся изменяемая копия в массиве, и программа печатает "Jello".

```
char s[] = "hello";
s[0] = 'J';
printf("%s\n", s);
```

[Источник: bug$_{huntpointersstrings}$.txt, BUG 5 + ключ]

P6. Возврат указателя на локальную переменную

```
char *greeting(void) {
    char buf[20];
    strcpy(buf, "Hi there");
    return buf;
}

int main(void) {
    char *g = greeting();
    printf("%s\n", g);
    return 0;
}
```

P6. Почему указатель g «висит» после вызова greeting()?

Ответ. buf живёт на стековом кадре greeting(), который исчезает сразу после возврата функции, — g становится висячим указателем (dangling); чтение *g — неопределённое поведение. Компиляторы

обычно предупреждают. Исправления по ключу: static-массив, буфер, переданный вызывающим, или malloc-память, которую вызывающий освободит. Вариант со static:

```
char *greeting(void) {
    static char buf[20];
    strcpy(buf, "Hi there");
    return buf;
}

int main(void) {
    char *g = greeting();
    printf("%s\n", g);
    return 0;
}
```

[Источник: bug$_{huntpointersstrings}$.txt, BUG 6 + ключ]

P7. Неинициализированный (дикий) указатель

```
int *p; // -> Memory address
*p = 10;
printf("%d\n", *p);
```

P7. Куда пишет *p = 10 и почему это опасно, даже если печатается 10?

Ответ. p никогда не присваивался — он содержит мусорный адрес, и *p = 10 пишет по неизвестному адресу: программа может упасть, а может «сработать» — опасный случай (в тестах ключа напечаталось 10 по чистой случайности). Исправление по ключу: указать p на реальную память — int v; int* p = &v; или p = malloc(sizeof(int));:

```
int v;
int *p = &v;
*p = 10;
printf("%d\n", *p);
```

[Источник: bug$_{huntpointersstrings}$.txt, BUG 7 + ключ]

P8. Передача по значению

```
void swap(int a, int b) {
    int t = a;
    a = b;
    b = t;
}

int main(void) {
    int x = 1, y = 2;
    swap(x, y);
    printf("%d %d\n", x, y);
    return 0;
}
```

P8. Почему после swap(x, y) печатаются "1 2", а не "2 1"?

Ответ. C передаёт копии: swap меняет местами собственные локальные копии x и y, переменные main остаются нетронутыми. Исправление по ключу — та же функция с указателями из лекции 7: void swap(int *a, int* b) и вызов swap(&x, &y):

```
void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}
```

```
int main(void) {
    int x = 1, y = 2;
    swap(&x;, &y;);
    printf("%d %d\n", x, y);
    return 0;
}
```

[Источник: bug$_{huntpointersstrings}$.txt, BUG 8 + ключ]

P9. malloc без +1 для '\0'

```
char *s = "hello";
char *copy = malloc(strlen(s));
strcpy(copy, s);
printf("%s\n", copy);
free(copy);
```

P9. На сколько байт переполняется heap-блок и почему?

Ответ. strlen не считает терминатор: для "hello" malloc(strlen(s)) выделяет 5 байт, а strcpy пишет 6 (5 букв + '\0') — переполнение heap-блока на один байт. Исправление по ключу: malloc(strlen(s) + 1).

```
char *s = "hello";
char *copy = malloc(strlen(s) + 1);
strcpy(copy, s);
printf("%s\n", copy);
free(copy);
```

[Источник: bug$_{huntpointersstrings}$.txt, BUG 9 + ключ]

P10. Беззнаковый счётчик в обратном цикле

```
char s[] = "abc";
for (size_t i = strlen(s) - 1; i >= 0; i--) {
    putchar(s[i]);
}
putchar('\n');
```

P10. Почему цикл «наоборот» не останавливается после 'a'?

Ответ. size$_t$ — беззнаковый тип: условие i >= 0 всегда истинно. После i = 0 декремент заворачивает i в огромное число, и s[i] читает далеко за пределами массива. Два исправления из ключа: знаковый счётчик или цикл с i > 0 и доступом s[i - 1]:

```
char s[] = "abc";
for (int i = (int)strlen(s) - 1; i >= 0; i--) {
    putchar(s[i]);
}
putchar('\n');

/* или: */
char t[] = "abc";
for (size_t i = strlen(t); i > 0; i--) {
    putchar(t[i - 1]);
}
putchar('\n');
```

[Источник: bug$_{huntpointersstrings}$.txt, BUG 10 + ключ]

P11. Пропущен & в scanf

```
int x;
scanf("%d", x);
```

```
printf("%d\n", x);
```

P11. Что делает scanf("%d", x) без &?

Ответ. scanf нужен адрес переменной; передача x (неинициализированного int) заставляет scanf трактовать его мусорное значение как указатель и писать по этому адресу — неопределённое поведение; большинство компиляторов предупреждает с -Wall. Исправление: scanf("%d", &x);.

```
int x;
scanf("%d", &x;);
printf("%d\n", x);
```

[Источник: bug$_{huntpointersstrings}$.txt, BUG 11 + ключ]

P12. Использование после free

```
int *p = malloc(sizeof(int));
*p = 42;
free(p);
printf("%d\n", *p);
```

P12. Почему печать *p после free(p) — ошибка?

Ответ. После free(p) память может быть уже переиспользована: чтение *p может напечатать 42, мусор или упасть (use after free — неопределённое поведение). По ключу: не трогать* p после free; печатать до освобождения; после free присвоить p = NULL, чтобы ошибка падала громко:

```
int *p = malloc(sizeof(int));
*p = 42;
printf("%d\n", *p);
free(p);
p = NULL;
```

[Источник: bug$_{huntpointersstrings}$.txt, BUG 12 + ключ]

Карта ошибок листа (из ключа)

Тип ошибки	Номера задач
Пропущен нуль-терминатор / +1 для '\0'	1, 4, 9
Путаница указателя и массива	2, 3
Память только для чтения / висячая память	5, 6, 12
Неинициализированные указатели	7, 11
Передача по значению	8
Беззнаковый оборот (wraparound)	10

Quick sort by bug type [Источник: bug$_{huntpointersstringsanswerkey}$.txt]

7. Шпаргалка: ключевые правила и таблицы

Финальная глава — всё самое нужное перед тестом и экзаменом: приоритет операторов, полная таблица форматов printf/scanf, ключевые правила C недель 4–7 одной страницей, десять ответов одной строкой и что дальше по курсу.

7.1. Приоритет операторов

Приоритет	Операторы	Комментарий
1 (наивысший)	++ – –, (cast), sizeof	унарные операторы, приведение типа, размер
2	*, /, %	мультипликативные — выше, чем + и –
3	+, –	аддитивные
4	==, !=	сравнение на равенство — выше, чем && и ||
5	&&, ||	логические
6 (низший)	=, +=, –=, *=, /=, %=	присваивание — очень низкий приоритет

Приоритет и порядок вычисления [Источник: lecture$_4$, слайд 25]

Правило слайда: используйте скобки ( ) щедро, чтобы избежать двусмысленностей и побочных эффектов приоритета. Примеры эквивалентности:

```
y = x*3+2                 /* same as y = (x*3)+2 */
x != 0 && y == 0          /* same as (x != 0) && (y == 0) */
d = c >= '0' && c <= '9'  /* same as d = (c >= '0') && (c <= '9') */
```

7.2. Форматы printf/scanf

Полная спецификация формата: **%[flags][width][.precision][length]<type>**. Спецификаторы типа (лекции 6 и 7):

Спецификатор	Тип	Пример / вывод
d, i	int	printf("%d",10); → 10
u	unsigned int	printf("%u",10); → 10
o	unsigned int (восьмеричное)	из лекции 7: u, o, x, X — unsigned int
x, X	unsigned int (шестнадцатеричное)	printf("%x",10); → 0xa
c	char	printf("%c",'A'); → A
s	string	printf("%s","hello"); → hello
f	float/double	printf("%f",2.3); → 2.3
e, E	вещественное (экспонента)	1e3, 1.2E3, 1E–3
F, g, G	double	из лекции 7: e, E, f, F, g, G — double
%	литеральный %	printf("%d %%",10); → 10%

Спецификаторы типа printf [Источник: Lecture$_6$, слайд 12; Lecture$_{71}$, слайд 3]

Модификатор	Смысл
h	интерпретировать как short; c i, d, o, u, x
l	интерпретировать как long; c i, d, o, u, x
L	интерпретировать как double; c e, f, g

Модификаторы длины [Источник: Lecture$_6$, слайд 16]

Вызов printf	Вывод
printf("%d",10)	«10»
printf("%4d",10)	«bb10» (b — пробел; width)

Вызов printf	Вывод
printf("%7s","hello")	«bbhello»
printf("%d,%+d,%+d",10,-10)	«10,+10,-10» (флаг +)
printf("%04d",10)	«0010» (флаг 0)
printf("%-7s","hello")	«hellobb» (флаг –: по левому краю)
printf("%.2f,%.0f",1.141,1.141)	«1.14,1» (precision)
printf("%.2e,%.0e",1.141,100.00)	«1.14e+00,1e+02»
printf("%.4s","hello")	«hell»
printf("%.1s","hello")	«h»

width, flags, precision [Источник: Lecture$_6$, слайды 13–15]

scanf — входной аналог printf с той же спецификацией; игнорирует пробельные символы; возвращает число прочитанных элементов или EOF; **аргументы — адреса переменных (кроме имён массивов)**:

printf	scanf
printf("%d",x)	scanf("%d",&x)
printf("%f",f)	scanf("%f",&f)
printf("%s",str)	scanf("%s",str) — & не нужен
printf("%s",str)	scanf("%20s",str) — & не нужен
printf("%s %s",fname,lname)	scanf("%20s %20s",fname,lname)

printf ↔ scanf [Источник: Lecture$_6$, слайд 20]

Символьные функции stdio.h: putchar(c) — символ в стандартный вывод (возвращает символ или EOF); getchar() — следующий символ стандартного ввода или EOF; getc(fp)/putc(c, fp) — то же для потока; getchar() — это getc(stdin), putchar(c) — putc(c, stdout). Построчно: fgets(line, maxlen, fp) читает строку (включая перевод строки), fputs(line, fp) пишет; в файлы: fopen(name, mode) с режимами "r", "w", "a" (плюс "b" для двоичных), fclose(fp); stdin/stdout/stderr — тоже FILE*.

7.3. Ключевые правила C

- **Типы операндов решают деление**: 4/3 → 1 (int), 4.0/3.0 → 1.333333; тип приёмника (double fraction = 4/3) не влияет — деление происходит до присваивания.
- **Имена переменных**: буквы, цифры и _; начинаются с буквы; ключевые слова нельзя; регистр важен (x и X — разные переменные).
- **Гарантии размеров**: sizeof(char) == 1; sizeof(char) < sizeof(short) <= sizeof(int) <= sizeof(long); unsigned имеет примерно вдвое больший диапазон; точность float — 6, double — 15 значащих цифр.
- **Истина и ложь**: любое ненулевое значение — истина, 0 — ложь; условие — числовое выражение или указатель; в C99 bool — из <stdbool.h>.
- **== — сравнение, = — присваивание**; сравнение == на float/double ненадёжно из-за конечной точности (0.1 не представимо точно).
- **Короткое замыкание**: && и || прекращают вычисление, когда результат определён; побочные эффекты правой части могут не выполниться.
- **break** выходит из самого внутреннего цикла или switch; **без break в case выполнение проваливается** в следующий case без новой проверки.

- **continue** пропускает остаток тела внутреннего цикла, переходя к условию (в for — к обновлению).
- **Точка с запятой после for(...)** делает тело цикла пустым — блок после неё выполнится один раз.
- **Аргументы передаются по значению**: чтобы функция изменила переменную вызывающего, передавайте указатель (swap(&x, &y)); возвращённая структура — отдельная копия, которую вызывающий решает, где сохранить.
- **Строка C — массив char с '\0'**: strlen не считает терминатор; для копирования нужно +1 байт (malloc(strlen(s) + 1)); %s читает до нулевого байта.
- **Строковые литералы только для чтения**: char *s = "hello"; s[0] = 'J'; — segfault; изменяемая копия — char s[] = "hello";.
- **scanf нужны адреса**: scanf("%d", &x); имя массива char уже адрес: scanf("%s", str).
- **Неинициализированные переменные — мусор**: локальная double discount без значения — undefined behavior; в C нет значений по умолчанию.
- **Рекурсия = базовый случай + меньшая задача + прогресс**: без прогресса ($count_{down}(n)$ вместо $count_{down}(n - 1)$) — бесконечная рекурсия и переполнение стека.

7.4. Десять ответов одной строкой

- 1. Что напечатает double fraction = 4/3 с %f? — 1.000000: целочисленное деление даёт 1 до преобразования.
- 2. Сколько значащих цифр у float и double? — 6 и 15 (FLT_{DIG}, DBL_{DIG}).
- 3. Високосный год? — year % 400 == 0 || (year % 4 == 0 && year % 100 != 0): 2000 — да, 1900 и 2100 — нет.
- 4. Что без break в switch? — проваливание в следующий case без проверки (мелодия 'A#' печатает C, затем D).
- 5. Сколько раз выполнится do-while? — хотя бы один: условие проверяется после тела.
- 6. Что делает continue? — пропускает остаток тела цикла и переходит к условию.
- 7. Как поменять местами x и y в main? — void swap(int *a, int* b) и вызов swap(&x, &y): по значению ничего не выйдет.
- 8. Что вернёт strcmp("aA", "aA")? — 0: строки равны; >0 — str1 больше, <0 — меньше.
- 9. Что печатает $count_{down}(3)$ и $count_{up}(3)$? — 3 2 1 и 1 2 3: печать до вызова — при спуске, после — при возврате.
- 10. fibonacci(4)? — 3; два базовых случая (F(0)=0, F(1)=1), fibonacci(2) вычисляется дважды.

7.5. Что дальше

По материалам лекций после недели 7 курс продолжается темами, объявленными прямо в слайдах: «More searching next week…» (лекция 7, слайд о бинарном поиске — больше о поиске), «More about sizeof() next time…» (слайд о sizeof) и «Implicit cast to/from void * also possible (more next week…)» (слайд о приведении указателей), а концепция модулей из лекции 5 переносится на внешние библиотеки («Same concept carries over to external libraries (next week…)»). Перед мидтермом повторите недели 4–7 по этому пособию в следующем порядке: типы и деление (глава 2) → условия, switch и циклы (глава 3) → функции, структуры и ввод-вывод (глава 4) → указатели, строки и рекурсия (глава 5), затем прогоните все bug hunt-ы (глава 6) и эту шпаргалку: три exit check недели 4 (почему 4/3 теряет дробь; почему 2100 — обычный год; почему граница идёт первой) и три exit check недели 7 (где останавливается факториал; почему n − 1 движется к базовому случаю; чего ждёт factorial(3)) — готовые вопросы для самопроверки.